

Design and Development of a Complete Database System for a Forensic Medical Department

Database System Report

Group 03 Members:

T. H. Abeywickrama	E/22/008
S. H. S. Hansara	E/22/130
H. T. D. Hatharasinghe	E/22/135
H. P. J. Gunawardhana	E/22/126

Lecturer:

Mr. Biswajith Dissanayake

University of Peradeniya

Faculty of Engineering

Department of Computer Engineering

CO2050 – Database Systems

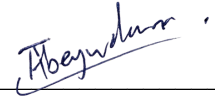
23rd July, 2026

Declaration

We hereby declare that this project report titled "Design and Development of a Complete Database System for a Forensic Medical Department" is a record of original work done by us under the guidance of our lecturers.

T. H. Abeywickrama

E/22/008



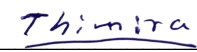
S. H. S. Hansara

E/22/130



H. T. D. Hatharasinghe

E/22/135



H. P. J. Gunawardhana

E/22/126



Acknowledgement

We would like to express our deepest appreciation to Dr. Chathula Wickramasinghe (Act. Consultant JMO) for providing domain expertise, documentation samples like MLEF and PMR, and insights into the practical workflows of the Forensic Medicine Department. We also thank our lecturer, Mr. Biswajith Dissanayake, and course instructors for their continuous guidance on database architecture and software engineering principles.

Abstract

The Department of Forensic Medicine handles highly sensitive and critical records, broadly categorised into Clinical Forensic components and Autopsy components. Currently, operations rely on a manual paper-based filing system, resulting in physical storage constraints, difficulty tracing pending cases (e.g., Medico-Legal Examination Forms and Postmortem Reports awaiting court dates), and challenges in preserving strict confidentiality. This project proposes the design and development of a secure, role-based computerised Database System to replace the manual process.

The system was developed using a modern technology stack: a PostgreSQL relational database for robust data management, a Node.js (Express) backend REST API for handling business logic, and a React (Vite) frontend with Tailwind CSS and Shaden UI for an intuitive user interface. The developed solution successfully digitises patient and deceased records, tracks laboratory requests and forensic evidence, auto-generates court-ready Medico-Legal Reports via a custom PDF library, and features a notification dashboard for pending cases. The system adheres to standard database normalisation principles and employs strict role-based access control to ensure medical data confidentiality. Ultimately, the system improves department efficiency, ensures accurate statistical reporting, and expedites the delivery of justice through timely court reporting.

Table of Contents

Declaration.....	1
Acknowledgement.....	2
Abstract.....	2
List of Figures.....	6
List of Tables.....	7
List of Abbreviations.....	8
Chapter 1 – Introduction.....	10
1.1 Background.....	10
1.2 Problem Statement.....	10
1.3 Objectives.....	10
1.4 Scope of the Project.....	11
1.5 Significance of the Study.....	11
1.6 Technologies Used.....	11
Chapter 2 – Requirement Analysis.....	12
2.1 Existing System.....	12
2.2 Problems in Existing System.....	12
2.3 Proposed System.....	12
2.4 Stakeholders.....	13
2.5 Functional Requirements.....	13
2.6 Non-functional Requirements.....	14
2.7 Use Cases.....	14
2.8 Workflow / Business Process.....	16
Chapter 3 – Database Design.....	17
3.1 Entity Identification.....	17
3.2 ER Diagram.....	18
3.3 Enhanced ER Diagram.....	19
3.4 Relational Schema.....	20
3.5 Data Dictionary.....	21
3.6 Primary Keys.....	25
3.7 Foreign Keys.....	25
3.8 Integrity Constraints.....	26
3.9 Normalisation.....	26
Chapter 4 – Database Implementation.....	28

4.1 DBMS Used.....	28
4.2 Database Creation.....	28
4.3 Table Creation.....	29
4.4 Constraints.....	33
4.5 Indexes.....	33
4.6 Views.....	34
4.7 Stored Procedures.....	34
4.8 Triggers.....	34
4.9 Sample Data.....	34
Chapter 5 – System Development.....	36
5.1 System Architecture.....	36
5.2 Login Module.....	36
5.3 Patient Management.....	37
5.4 Case Management.....	37
5.5 Postmortem Module.....	38
5.6 Evidence Management.....	38
5.7 Court Report Module.....	39
5.8 Staff Management.....	39
5.9 Report Generation.....	40
5.10 Dashboard.....	41
5.11 User Authentication.....	41
Chapter 6 – Testing.....	42
6.1 Test Plan.....	42
6.2 Test Cases.....	42
6.3 Validation Testing.....	43
6.4 SQL Query Testing.....	43
6.5 Backup and Recovery.....	44
Chapter 7 – Reports.....	45
Chapter 8 – Security.....	50
Chapter 9 – Challenges and Future Improvements.....	52
Challenges.....	52
Future Improvements.....	52
Chapter 10 – Conclusion.....	53
References.....	54
Appendices.....	55
Appendix A – Complete SQL Schema Script (schema.sql).....	55

Appendix B – Source Code Structure.....	59
Appendix C – Docker Configuration.....	60
Individual Contribution.....	61
Project Timeline.....	62
Checklist Before Submission.....	63

List of Figures

Figure 1: Use Case Diagrams.....	15
Figure 2: Workflow Diagram.....	16
Figure 3: ER Diagram.....	18
Figure 4: EER Diagram.....	19
Figure 5: Relational Schema Diagram.....	20
Figure 6: Login Page.....	36
Figure 7: Patient Registration Page.....	37
Figure 8: Patient Records Page.....	37
Figure 9: PMR Form Page.....	38
Figure 10: Lab Records Page.....	38
Figure 11: MLR Page.....	39
Figure 12: Staff Management Page.....	39
Figure 13: Daily Case Reports Page.....	40
Figure 14: Monthly Statistics Page.....	40
Figure 15: Dashboard Page.....	41
Figure 16: MLEF Form Report.....	45
Figure 17: PMR Form Report.....	46
Figure 18: MLR Form Report.....	47
Figure 19: Daily Case Report.....	48
Figure 20: Monthly Statistics Report.....	49

List of Tables

Table 1: List of Abbreviations.....	8
Table 2: Technologies used.....	11
Table 3: Users Table.....	21
Table 4: Patients Table.....	22
Table 5: Mlef_forms Table.....	22
Table 6: Mlr_reports Table.....	23
Table 7: Mlr_injuries Table.....	23
Table 8: Mlr_grievous_entries Table.....	23
Table 9: Pmr_forms Table.....	24
Table 10: Pmr_identifiers Table.....	24
Table 11: Lab_requests Table.....	24
Table 12: Test Cases.....	42
Table 13: Individual contributions.....	61
Table 14: Project timeline.....	62

List of Abbreviations

Table 1: List of Abbreviations

BHT	Bed Head Ticket
COD	Cause of Death
ERD/ER	Entity-Relationship Diagram
EERD/EER	Enhanced Entity-Relationship Diagram
DBMS	Database Management System
JMO	Judicial Medical Officer
MLEF	Medico-Legal Examination Form
MLR	Medico-Legal Report
PMR	Postmortem Report
RBAC	Role-Based Access Control
SQL	Structured Query Language
SRS	Software Requirement Specification
NIC	National Identity Card
SLMC	Sri Lanka Medical Council
PK	Primary Key
FK	Foreign Key
1NF / 2NF / 3NF	First / Second / Third Normal Form
API	Application Programming Interface
UI	User Interface
JSON	JavaScript Object Notation
JWT	JSON Web Tokens
REST	Representational State Transfer

SPA	Single Page Application
PDF	Portable Document Format
PITR	Point-in-Time Recovery
CI/CD	Continuous Integration / Continuous Deployment

Chapter 1 – Introduction

1.1 Background

The Department of Forensic Medicine is a critical junction between the medical and legal fields. It handles two major components: Clinical Forensics (dealing with living individuals, such as trauma victims, domestic/sexual abuse victims, and detainees) and Autopsies (dealing with hospital and non-hospital deaths). Currently, the department relies heavily on manual record-keeping. Forms such as the Medico-Legal Examination Form (MLEF) and the Postmortem Report (PMR) are filled out by hand and stored physically in secure record rooms and filing cabinets.

1.2 Problem Statement

The reliance on a manual documentation system presents several critical issues:

- **Paper Records:** Physical documents are prone to wear, tear, misplacement, and require massive physical storage space.
- **Difficulty in Searching:** Tracing specific historical cases, matching suspects, or finding old autopsy reports takes immense manual effort.
- **Confidentiality:** Physical files passed between clerks, laboratory staff, and doctors risk exposing highly sensitive patient information.
- **Report Generation:** Monitoring pending reports (e.g., unissued MLEFs, pending court dates) is inefficient. Doctors lack a unified dashboard to see their outstanding legal obligations.
- **Evidence Tracking:** Tracing the chain of custody for forensic samples (blood, swabs) sent for external investigations is currently fragmented.

1.3 Objectives

- **General Objective:** To design and develop a comprehensive, secure, and user-friendly database management system to digitise the workflows of the Forensic Medical Department.
- **Specific Objectives:**
 - To digitise the creation, storage, and retrieval of MLEF, MLR, and PMR documents.
 - To implement Role-Based Access Control (RBAC) to enforce strict data confidentiality.
 - To automate the generation of court-ready documents (MLR and PMR) directly from the database using a dedicated PDF module.
 - To provide notification dashboards for doctors to track pending reports and upcoming court dates.
 - To track the status of laboratory investigations and evidence processing.

1.4 Scope of the Project

The system encompasses the internal workflow of the Forensic Medicine Department. It includes patient registration, deceased registration, case management (clinical and autopsy), laboratory request tracking, and statistical report generation. It has dashboards and processes allocated for the use of staff, and explicitly excludes external system integrations (e.g., direct connections to Police databases) due to strict security boundaries.

1.5 Significance of the Study

Implementing this system will drastically reduce the administrative burden on Judicial Medical Officers (JMOs) and clerical staff, prevent the loss of crucial judicial evidence, expedite court proceedings by ensuring timely report generation, and allow for robust statistical analysis of regional crime and mortality rates.

1.6 Technologies Used

Table 2: Technologies used

Database	PostgreSQL (Relational Database Management System)
Backend	Node.js with Express.js framework
Frontend	React.js, compiled with Vite, built with TypeScript
Styling & UI	Tailwind CSS and Shadcn UI component library
Testing	Jest (Backend testing)
Storage	Supabase (Object storage such as profile pictures and documents)
Deployment	Docker (For unified containerisation)

Chapter 2 – Requirement Analysis

2.1 Existing System

In the existing manual process, patients or bodies arrive with a referral (ward referral, police order, or court order). A manual registry is updated. Doctors perform the examination and fill out the MLEF or PMR by hand based on BHTs, crime scene photos, and witness statements. If investigations (X-rays, toxicology, histology) are required, paper request forms are dispatched. Once all data is collected, a final Cause of Death (COD) report or Medico-Legal Report (MLR) is manually drafted, signed, and dispatched.

2.2 Problems in Existing System

- **Inefficient Tracking:** The manual flow lacks real-time status tracking. Doctors often lose track of which MLEFs are pending lab results, and there is no automated alert system for approaching court dates, leading to delayed legal proceedings.
- **Error-Prone Reporting:** Compiling daily, monthly, and annual statistical reports requires manually counting ledger entries. This process is time-consuming, tedious, and highly susceptible to human error.
- **Slow Data Retrieval:** Searching for historical case files or patient histories requires manually sifting through physical filing cabinets, which slows down active investigations.
- **Security & Integrity Risks:** Physical files are vulnerable to loss, damage, or unauthorised access. Furthermore, handwritten notes on forms are prone to legibility issues, which can cause critical misinterpretations of medical evidence.

2.3 Proposed System

- **Centralised Digital Dashboard:** The proposed system centralises all records into a single secure database. An intuitive dashboard instantly alerts doctors and JMOs to pending cases, upcoming court dates, and recently updated lab results.
- **Standardised Data Entry:** Data entry is streamlined through specific, structured frontend forms. This eliminates the risks associated with illegible handwriting and ensures all legally required fields are captured.
- **Real-Time Digital Tracking:** Lab requests and evidence chain-of-custody are tracked digitally via dedicated modules. Staff can instantly view the real-time status of pending tests without needing manual follow-ups.
- **Automated Report Generation:** Instead of manual compilation, final medico-legal reports and monthly statistical charts are automatically generated as standardised, printable PDFs directly from the verified database entries, saving time and eliminating counting errors.

- **Role-Based Security:** The system implements strict role-based access controls, ensuring that sensitive medical and forensic data is only accessible to authorised personnel, thereby improving overall data security and integrity.

2.4 Stakeholders

- Patients & Relatives of the Deceased
- Judicial Medical Officers (JMOs)
- Doctors
- Laboratory Staff
- Administrative Staff
- Police Officers & Court Officials
- System Owner

2.5 Functional Requirements

- **Authentication:** The system must securely authenticate user's identity before granting access to system features based on their specific role and permissions.
- **Patient & Deceased Management:** The system must capture and record personal information and referral details.
- **Clinical Module:** The system must allow doctors to input and manage MLEF data and then generate final MLR.
- **Autopsy Module:** The system must allow JMOs to input and manage PMR data and then record cause of death.
- **Laboratory Module:** The system must allow officers to issue and track investigation requests and lab reports.
- **Dashboard & Notifications:** The system must display important information and track pending reports and court dates per doctor/JMO.
- **Report Generation:** The system must allow authorised users to automatically compile medical findings, evidence logs, and case notes into standardised, legally compliant, and exportable documents for court proceedings or official archives.
- **Manage Evidence:** The system must provide a secure process to log physical and digital items, accurately track the chain of custody, and attach related files.
- **Search Records:** The system must provide a robust search and filtering tool allowing users to rapidly locate and retrieve specific case files, patient histories, or evidence details from the database.

2.6 Non-functional Requirements

- **Security:** The system must enforce strict role-based access controls and robust encryption (both at rest and in transit) to protect highly sensitive records from unauthorised access.
- **Performance:** The system must be highly responsive, capable of quickly processing, uploading, and retrieving large files without lagging during critical investigations.
- **Availability:** The system must maintain an optimum uptime, ensuring it is accessible 24/7 to support continuous operations in the department.
- **Confidentiality:** The system must guarantee that all patient identities, medical histories, and ongoing investigation details are strictly anonymised where necessary and only visible to authorised personnel.
- **Reliability:** The system must consistently function without data corruption or unexpected failures, ensuring that all forensic evidence and medical logs are accurately recorded, preserved, and legally admissible.
- **Usability:** The system must feature an intuitive, logically organised interface that allows users to efficiently input, search, and analyse data without requiring extensive technical training.

2.7 Use Cases

- **Create Case:** The JMO logs in, selects a registered patient/deceased, and initiates a new MLEF/PMR record.
- **Manage Lab Requests:** The Doctor links a lab request to a case. Lab Staff log in, view pending requests, and update the status once results are ready.
- **Register Patient/Deceased:** The doctor or JMO logs in, navigates to the registration section, and enters the details to create a new patient or deceased subject record in the system.
- **Complete Medico-Legal Examination (MLEF):** The doctor logs in, selects an existing patient record, and fills out the MLEF with injury details, history, and examination findings, then saves the form.
- **Document Post-Mortem Report (PMR):** The JMO selects a deceased subject from the registry, initiates a new Post Mortem Report, enters internal/external examination findings, and records the cause of death.
- **Register Staff Member:** The administrative staff logs into the system and navigates to the staff registration page to create a new account for a doctor, JMO, or lab technician, assigning them appropriate role-based permissions.
- **Issue Medico-Legal Report (MLR):** The Doctor reviews a completed MLEF file, finalises the legal and medical conclusions, and issues the final Medico-Legal Report for court use.
- **View Statistics & Daily Reports:** The administrative staff logs into the dashboard to view daily activity logs or generate monthly statistical charts on the number of cases, autopsies, and lab requests processed.



Figure 1: Use Case Diagrams

2.8 Workflow / Business Process

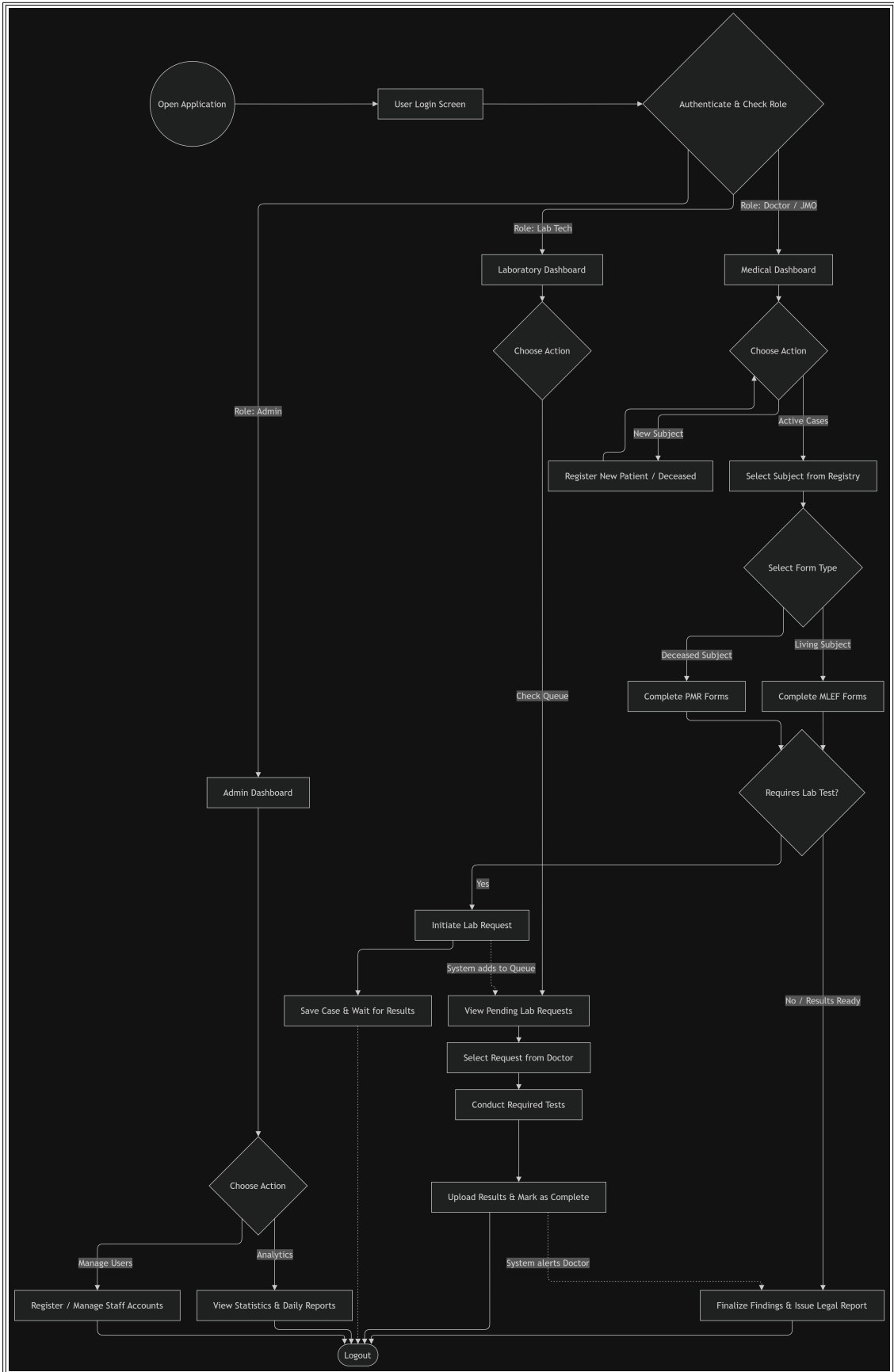


Figure 2: Workflow Diagram

Chapter 3 – Database Design

3.1 Entity Identification

Based on the provided database schema, the system consists of the following primary and weak/child entities:

- **User** (Represents Doctors, Admins, JMOs, Lab Techs)
- **Patient** (Individuals undergoing medico-legal examinations)
- **MLEF Form** (Medico-Legal Examination Form – Parts A and B)
- **MLR Report** (Medico-Legal Report)
- **MLR Injury** (Child entity of MLR Report detailing specific injuries)
- **MLR Grievous Entry** (Child entity of MLR Report for grievous hurts)
- **PMR Form** (Post-Mortem Report)
- **PMR Identifier** (Child entity of PMR Form for identification details)
- **Lab Request** (Laboratory testing requests and results)

For ER, EER, and Schema Diagrams, please proceed to the next pages →

3.2 ER Diagram

Please zoom in for a better view or use the separate diagram files given in the DVD.

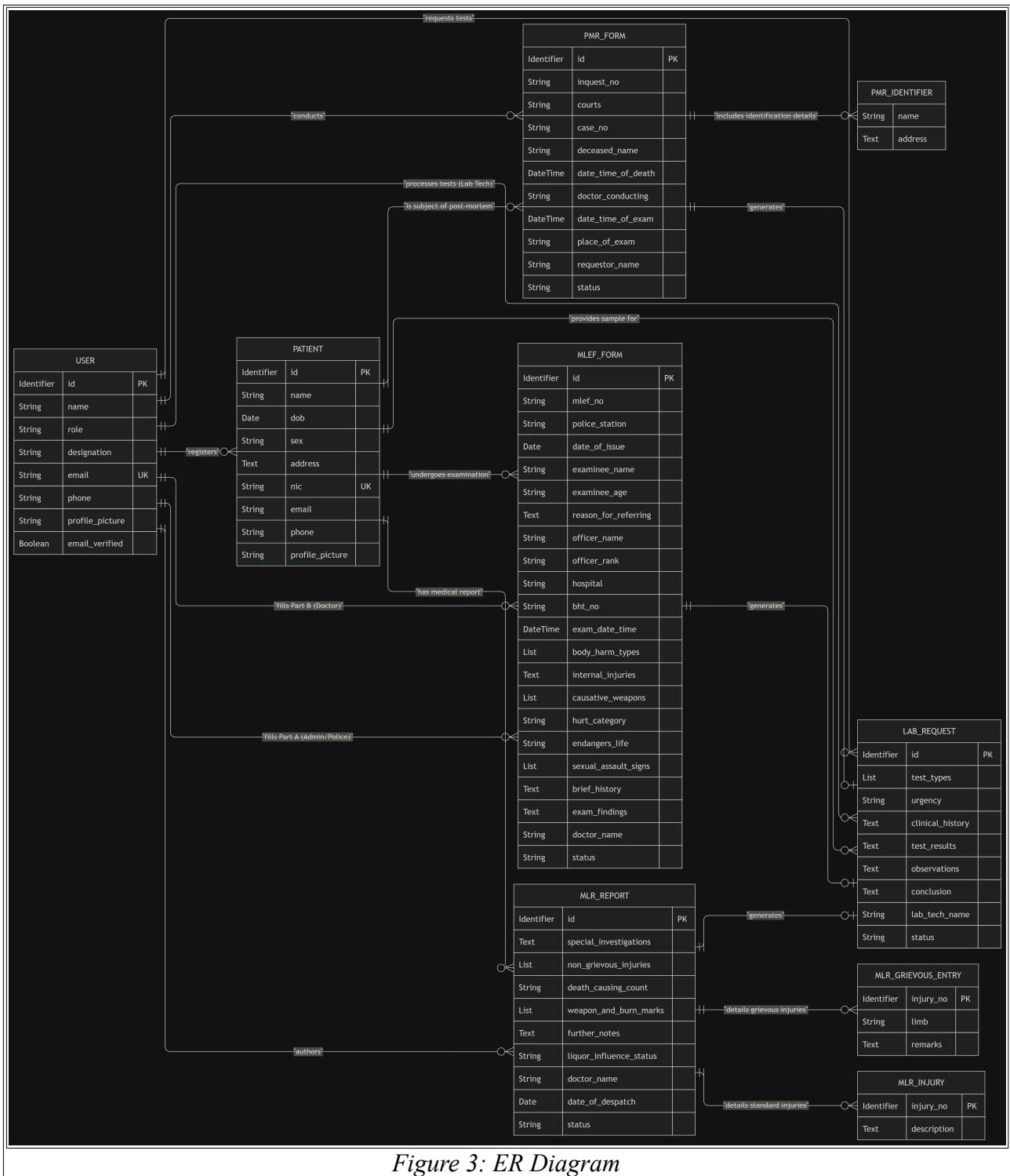


Figure 3: ER Diagram

3.3 Enhanced ER Diagram

Please zoom in for a better view or use the separate diagram files given in the DVD.

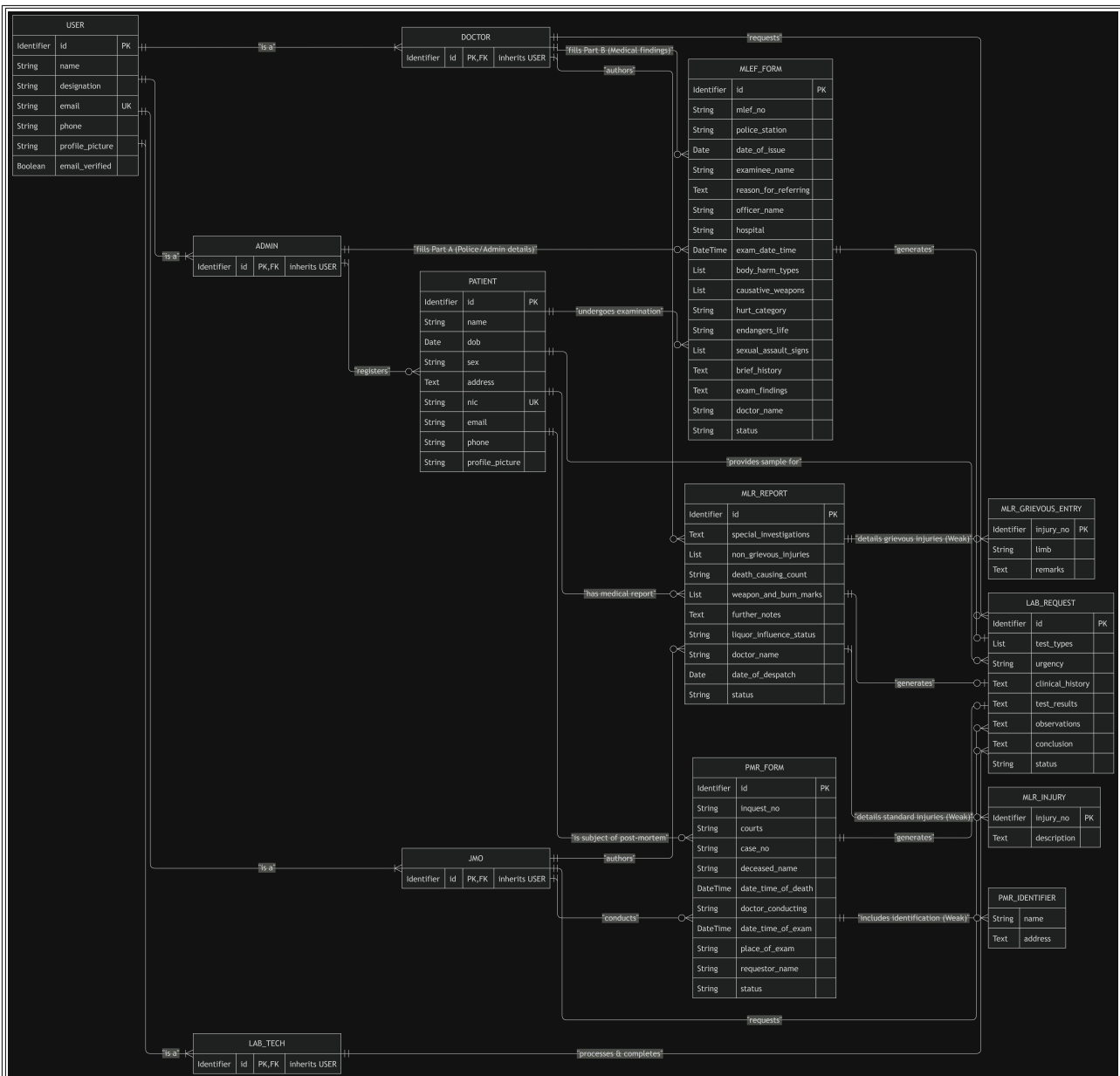


Figure 4: EER Diagram

3.4 Relational Schema

Please zoom in for a better view or use the separate diagram files given in the DVD.



Figure 5: Relational Schema Diagram

3.5 Data Dictionary

Table 3: Users Table

Attribute	Data Type	Size	Description	Constraints
id	VARCHAR	50	Unique identifier for the user	PRIMARY KEY
name	VARCHAR	255	Full name of the user	NOT NULL
role	VARCHAR	50	System role (doctor, jmo, admin, lab)	NOT NULL
designation	VARCHAR	255	Professional designation	NOT NULL
email	VARCHAR	255	User's email address	UNIQUE, NOT NULL
phone	VARCHAR	50	Contact phone number	NOT NULL
password_hash	VARCHAR	255	Encrypted password	NOT NULL
profile_picture_url	VARCHAR	500	URL to user's profile image	None
email_verified	BOOLEAN	1 byte	Indicates if email is confirmed	NOT NULL, DEFAULT FALSE
verification_code_hash	VARCHAR	255	Hashed 6-digit verification code	None
verification_expires_at	TIMESTAMP	8 bytes	Expiry time for verification	None
verification_sent_at	TIMESTAMP	8 bytes	Throttle control for resends	None
verification_attempts	INTEGER	4 bytes	Count of verification attempts	NOT NULL, DEFAULT 0
reset_code_hash	VARCHAR	255	Hashed password reset code	None
reset_expires_at	TIMESTAMP	8 bytes	Expiry time for reset code	None
reset_sent_at	TIMESTAMP	8 bytes	Throttle control for reset emails	None
reset_attempts	INTEGER	4 bytes	Count of reset attempts	NOT NULL, DEFAULT 0
created_at	TIMESTAMP	8 bytes	Record creation timestamp	DEFAULT CURRENT_TIMESTAMP

Table 4: Patients Table

Attribute	Data Type	Size	Description	Constraints
id	VARCHAR	50	Unique identifier for the patient	PRIMARY KEY
name	VARCHAR	255	Full name of the patient	NOT NULL
dob	DATE	4 bytes	Date of birth	NOT NULL
sex	VARCHAR	50	Biological sex	NOT NULL
address	TEXT	Variable	Residential address	NOT NULL
nic	VARCHAR	50	National Identity Card number	UNIQUE, NOT NULL
email	VARCHAR	255	Email address	NOT NULL
phone	VARCHAR	50	Contact phone number	NOT NULL
registered_by	VARCHAR	255	Name of the registering user	NOT NULL
profile_picture_url	VARCHAR	500	URL to patient's photo	None
registered_at	TIMESTAMP	8 bytes	Timestamp of registration	DEFAULT CURRENT_TIME STAMP

Table 5: Mlef_forms Table

Attribute	Data Type	Size	Description	Constraints
id	VARCHAR	50	Unique identifier for the form	PRIMARY KEY
patient_id	VARCHAR	50	Associated patient	FK (patients.id), ON DELETE CASCADE
(Part A Fields)	Various	Various	Fields for Police/Admin (station, officer details, etc.)	None
body_harm_types	TEXT[]	Array	Array of harm types	None
causative_weapon	TEXT[]	Array	Array of causative weapons	None
sexual_assault_signs	TEXT[]	Array	Array of assault signs	None
(Part B Fields)	Various	Various	Doctor exam findings, remarks, etc.	None
part_b_filled_by	VARCHAR	50	User who filled Part B	FK (users.id)
part_b_filled_at	TIMESTAMP	8 bytes	Timestamp of Part B completion	None
lab_request_id	VARCHAR	50	Linked lab request identifier	None
status	VARCHAR	50	Current state of the form	DEFAULT 'draft'
created_at	TIMESTAMP	8 bytes	Record creation timestamp	DEFAULT CURRENT_TIM ESTAMP

created_by	VARCHAR	50	Creator of the record	FK (users.id)
<i>(Note: Part A and B standard text/varchar fields are truncated for simplicity here. They share standard data types and no specific constraints.)</i>				

Table 6: Mlr_reports Table

Attribute	Data Type	Size	Description	Constraints
id	VARCHAR	50	Unique identifier for the MLR	PRIMARY KEY
patient_id	VARCHAR	50	Associated patient	FK (patients.id), ON DELETE CASCADE
non_grievous_nos	TEXT[]	Array	Array of non-grievous injury numbers	None
<i>(Various Stats)</i>	VARCHAR	255	Death causing counts, blunt weapon nos	None
date_of_despatch	DATE	4 bytes	Date report was despatched	None
status	VARCHAR	50	Current state of the report	DEFAULT 'draft'
created_by	VARCHAR	50	Creator of the record	FK (users.id)

Table 7: Mlr_injuries Table

Attribute	Data Type	Size	Description	Constraints
id	SERIAL	4 bytes	Auto-incrementing identifier	PRIMARY KEY
mlr_id	VARCHAR	50	Associated MLR report	FK (mlr_reports.id), ON DELETE CASCADE
injury_no	VARCHAR	50	Designated number of the injury	NOT NULL
description	TEXT	Variable	Detailed description of injury	NOT NULL

Table 8: Mlr_grievous_entries Table

Attribute	Data Type	Size	Description	Constraints
id	VARCHAR	50	Unique identifier for the entry	PRIMARY KEY
mlr_id	VARCHAR	50	Associated MLR report	FK (mlr_reports.id), ON DELETE CASCADE
injury_no	VARCHAR	50	Associated injury number	None
limb	VARCHAR	255	Affected limb	None
remarks	TEXT	Variable	Additional clinical remarks	None

Table 9: Pmr_forms Table

Attribute	Data Type	Size	Description	Constraints
id	VARCHAR	50	Unique identifier for PMR	PRIMARY KEY
patient_id	VARCHAR	50	Associated patient/deceased	FK (patients.id), ON DELETE CASCADE
(PMR Details)	Various	Various	Inquest, dates, location, doctor details	None
status	VARCHAR	50	Current state of the report	DEFAULT 'draft'
created_by	VARCHAR	50	Creator of the record	FK (users.id)

Table 10: Pmr_identifiers Table

Attribute	Data Type	Size	Description	Constraints
id	SERIAL	4 bytes	Auto-incrementing identifier	PRIMARY KEY
pmr_id	VARCHAR	50	Associated Post-Mortem form	FK (pmr_forms.id), ON DELETE CASCADE
name	VARCHAR	255	Name of the identifier person	NOT NULL
address	TEXT	Variable	Address of the identifier person	NOT NULL

Table 11: Lab_requests Table

Attribute	Data Type	Size	Description	Constraints
id	VARCHAR	50	Unique identifier for request	PRIMARY KEY
patient_id	VARCHAR	50	Associated patient	FK (patients.id), ON DELETE CASCADE
requested_by	VARCHAR	50	User who ordered the lab test	FK (users.id)
form_type	VARCHAR	50	Type of source form	NOT NULL
form_id	VARCHAR	50	ID of the source form	NOT NULL
test_types	TEXT[]	Array	Array of required tests	None
urgency	VARCHAR	50	Priority level	NOT NULL
status	VARCHAR	50	Current state of test	DEFAULT 'pending'
completed_at	TIMESTAMP	8 bytes	Timestamp of completion	None

3.6 Primary Keys

Primary keys (PK) enforce entity integrity by uniquely identifying each row in a table.

- users table: id
- patients table: id
- mlef_forms table: id
- mlr_reports table: id
- mlr_injuries table: id
- mlr_grievous_entries table: id
- pmr_forms table: id
- pmr_identifiers table: id
- lab_requests table: id

3.7 Foreign Keys

Foreign keys (FK) maintain referential integrity between linked entities.

- mlef_forms.patient_id references patients.id
- mlef_forms.part_b_filled_by references users.id
- mlef_forms.created_by references users.id
- mlr_reports.patient_id references patients.id
- mlr_reports.created_by references users.id
- mlr_injuries.mlr_id references mlr_reports.id
- mlr_grievous_entries.mlr_id references mlr_reports.id
- pmr_forms.patient_id references patients.id
- pmr_forms.created_by references users.id
- pmr_identifiers.pmr_id references pmr_forms.id
- lab_requests.patient_id references patients.id
- lab_requests.requested_by references users.id

3.8 Integrity Constraints

- **Entity Integrity:** Enforced via PRIMARY KEY on all tables, ensuring no duplicate or null identifiers.
- **Referential Integrity:** Enforced via FOREIGN KEY constraints. The use of ON DELETE CASCADE on patient and report IDs ensures that if a parent record (like a patient or MLR report) is deleted, all associated child records (like injuries or PMR identifiers) are automatically deleted to prevent orphaned data.
- **Domain Integrity:** Maintained through strict data typing (VARCHAR, DATE, TIMESTAMP, BOOLEAN, TEXT[], SERIAL).
- **Unique Constraints (UNIQUE):**
 - users.email ensures no two users share an email account.
 - patients.nic ensures a National Identity Card is only registered once.
- **Not Null Constraints (NOT NULL):** Prevents essential fields from being left blank (e.g., patients.dob, users.role, mlr_injuries.description).
- **Default Constraints (DEFAULT):** Assigns automatic baseline values (e.g., status DEFAULT 'draft', email_verified DEFAULT FALSE, created_at DEFAULT CURRENT_TIMESTAMP).

3.9 Normalisation

Normalisation is the process of organising data in a database to reduce redundancy and improve data integrity. Here is how the schema progresses through the Normal Forms:

Unnormalized Form (UNF)

In a hypothetical UNF state, the database would consist of a massive, flat table where all data is grouped together.

A single row for an examination would contain: Patient Details (Name, NIC, DOB), Doctor Details (Name, Role, SLMC No), Form Details, and a repeating group of injuries (Injury 1, Injury 2, Injury 3) and lab tests.

First Normal Form (1NF)

Eliminate repeating groups. Every column must hold atomic (indivisible) values, and each record must be unique.

How it is applied:

- In the transition to 1NF, repeating groups were separated into their own distinct entities.
- Instead of storing multiple injuries in the mlr_reports table as injury_1, injury_2, etc., a new child table mlr_injuries was created. Each injury now gets its own row.
- Similarly, pmr_identifiers was created to handle instances where multiple people identify a body.
- However, the schema utilizes PostgreSQL's native array types e.g., TEXT[] for body_harm_types and test_types. While classical strict 1NF frowns upon multi-valued attributes,

modern RDBMS arrays provide an optimised way to store discrete categorical tags without full table separation.

Second Normal Form (2NF)

Must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key (eliminate partial dependencies).

How it is applied:

- This is evident in the child tables.
- If a composite primary key was used in the mlr_injuries table like (mlr_id, injury_no), a violation of 2NF would occur if we stored the MLR's date_of_despatch in this table, because the date only depends on mlr_id, not the injury_no.
- The current schema satisfies 2NF by using surrogate primary keys (id SERIAL) and keeping the report-level details (like despatch date and status) strictly inside the parent mlr_reports table.
- Attributes in mlr_injuries (description, injury_no) rely entirely on the injury entity itself.

Third Normal Form (3NF)

Must be in 2NF, and no non-key attribute can depend on another non-key attribute (eliminate transitive dependencies).

How it is applied:

- The primary application of 3NF is the separation of users and patients from the form tables (mlef_forms, pmr_forms).
- If patients was not a separate table, a transitive dependency would exist in mlef_forms: mlef_forms.id → patient_id → patient_address. To update a patient's address, you would have to update every historical form they ever had, risking update anomalies.
- By placing patient demographics in the patients table and linking it via a Foreign Key (patient_id), the schema adheres to 3NF.
- However, fields like doctor_name and slmc_reg_no in mlef_forms represent a deliberate denormalisation (violating strict 3NF). In Medico-Legal systems, preserving the exact qualifications and name of a doctor at the time the form was signed is legally crucial, preventing historical records from changing if the doctor updates their profile later.

Chapter 4 – Database Implementation

4.1 DBMS Used

The database management system chosen for this Medico-Legal Records Management System is PostgreSQL. PostgreSQL is an advanced, enterprise-class open-source relational database that supports both SQL (relational) and JSON (non-relational) querying. It was selected for its robustness, performance, reliability, and strong support for complex queries, data integrity, and concurrent transactions, which are critical for handling sensitive medical and legal records. The system connects to the database using the pg node module.

4.2 Database Creation

The database schema is initialised using a Node.js script that reads the schema.sql file and executes the SQL commands against the connected PostgreSQL instance. The application uses connection pooling to manage database connections efficiently:

```
// db.js (Connection Configuration snippet)
import pkg from 'pg';
import dotenv from 'dotenv';

dotenv.config();
const { Pool } = pkg;

const pool = new Pool({
  connectionString: process.env.POSTGRES_URI,
  ssl: {
    rejectUnauthorized: false
  }
});

export default pool;

// init_db.js (Execution Script snippet)
async function initDatabase() {
  try {
    const schemaPath = path.join(__dirname, 'schema.sql');
    const sql = fs.readFileSync(schemaPath, 'utf8');
    await pool.query(sql);
    console.log("Database initialized successfully with all tables!");
  } catch (error) {
    console.error("Failed to initialize database:", error);
  }
}
```

4.3 Table Creation

The database consists of 9 primary tables to manage users, patients, forms, and laboratory requests. Below are the SQL scripts used to create these tables, as defined in schema.sql:

```
-- 1. Users Table (Doctors, Admins, JMOs, Lab Techs)
CREATE TABLE users (
    id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL, -- 'doctor', 'jmo', 'admin', 'lab'
    designation VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(50) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    profile_picture_url VARCHAR(500),
    -- Email verification: a user must confirm their address once before first
login
    email_verified BOOLEAN NOT NULL DEFAULT FALSE,
    verification_code_hash VARCHAR(255), -- bcrypt hash of the 6-digit
code, never the code itself
    verification_expires_at TIMESTAMP,
    verification_sent_at TIMESTAMP, -- used to throttle resend
requests
    verification_attempts INTEGER NOT NULL DEFAULT 0,
    -- Password reset: same emailed-code mechanism as verification, kept in
-- separate columns so a reset in progress cannot clash with a verification
    reset_code_hash VARCHAR(255),
    reset_expires_at TIMESTAMP,
    reset_sent_at TIMESTAMP,
    reset_attempts INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Patients Table
CREATE TABLE patients (
    id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    dob DATE NOT NULL,
    sex VARCHAR(50) NOT NULL,
    address TEXT NOT NULL,
    nic VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    registered_by VARCHAR(255) NOT NULL, -- name of user
    profile_picture_url VARCHAR(500),
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. MLEF Forms (Medico-Legal Examination Forms)
CREATE TABLE mlef_forms (
    id VARCHAR(50) PRIMARY KEY,
    patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,
```

```

-- Part A (Police / Admin)
police_station VARCHAR(255),
mlef_no VARCHAR(255),
date_of_issue DATE,
examinee_name VARCHAR(255),
examinee_address TEXT,
examinee_age VARCHAR(50),
examinee_sex VARCHAR(50),
reason_for_referring TEXT,
officer_name VARCHAR(255),
officer_rank VARCHAR(255),
officer_reg_no VARCHAR(255),
officer_police_station VARCHAR(255),
part_a_filled_by VARCHAR(255),
part_a_filled_at TIMESTAMP,

-- Part B (Medical Officer)
hospital VARCHAR(255),
ward VARCHAR(255),
bht_no VARCHAR(255),
admission_date DATE,
exam_date_time TIMESTAMP,
discharge_date DATE,
exam_place VARCHAR(255),
body_harm_types TEXT[], -- Array of strings e.g. ['laceration',
'contusion']
internal_injuries TEXT,
causative_weapon TEXT[], -- Array of strings
causative_weapon_other TEXT,
hurt_category VARCHAR(50),
endangers_life VARCHAR(50),
alcohol_exam VARCHAR(50),
drugs_exam VARCHAR(50),
sexual_assault_signs TEXT[], -- Array of strings
brief_history TEXT,
exam_findings TEXT,
investigations TEXT,
referrals TEXT,
other_opinions TEXT,
remarks TEXT,
doctor_name VARCHAR(255),
doctor_qualifications VARCHAR(255),
slmc_reg_no VARCHAR(255),
doctor_designation VARCHAR(255),
ref_no VARCHAR(255),
part_a_pdf_url VARCHAR(500),
part_b_pdf_url VARCHAR(500),
part_b_filled_by VARCHAR(50) REFERENCES users(id),
part_b_filled_at TIMESTAMP,

lab_request_id VARCHAR(50), -- Linked lab request
status VARCHAR(50) DEFAULT 'draft', -- 'draft', 'complete'
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```

        created_by VARCHAR(50) REFERENCES users(id)
    );

-- 4. MLR Reports (Medico-Legal Reports)
CREATE TABLE mlr_reports (
    id VARCHAR(50) PRIMARY KEY,
    patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,
    special_investigations TEXT,
    non_grievous_nos TEXT[], -- Array of injury numbers
    death_causing_count VARCHAR(255),
    blunt_weapon_nos VARCHAR(255),
    blunt_contusion_nos VARCHAR(255),
    cut_nos VARCHAR(255),
    sharp_cutting_nos VARCHAR(255),
    stab_nos VARCHAR(255),
    firearms_nos VARCHAR(255),
    burns_nos VARCHAR(255),
    bite_nos VARCHAR(255),
    further_notes TEXT,
    patient_smell_liquor VARCHAR(50),
    under_influence_liquor VARCHAR(50),
    doctor_name VARCHAR(255),
    doctor_qualifications VARCHAR(255),
    designation VARCHAR(255),
    station VARCHAR(255),
    date_of_despatch DATE,
    lab_request_id VARCHAR(50),
    pdf_url VARCHAR(500),
    status VARCHAR(50) DEFAULT 'draft', -- 'draft', 'submitted'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_by VARCHAR(50) REFERENCES users(id)
);

-- 5. MLR Injuries Table (Child table for MLR Report)
CREATE TABLE mlr_injuries (
    id SERIAL PRIMARY KEY,
    mlr_id VARCHAR(50) REFERENCES mlr_reports(id) ON DELETE CASCADE,
    injury_no VARCHAR(50) NOT NULL,
    description TEXT NOT NULL
);

-- 6. MLR Grievous Entries Table (Child table for MLR Report)
CREATE TABLE mlr_grievous_entries (
    id VARCHAR(50) PRIMARY KEY,
    mlr_id VARCHAR(50) REFERENCES mlr_reports(id) ON DELETE CASCADE,
    injury_no VARCHAR(50),
    limb VARCHAR(255),
    remarks TEXT
);

-- 7. PMR Reports (Post-Mortem Reports)
CREATE TABLE pmr_forms (
    id VARCHAR(50) PRIMARY KEY,

```

```

patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,
inquest_no VARCHAR(255),
place VARCHAR(255),
courts VARCHAR(255),
date DATE,
case_no VARCHAR(255),
deceased_name VARCHAR(255),
date_time_of_death TIMESTAMP,
doctor_conducting VARCHAR(255),
date_time_of_exam TIMESTAMP,
place_of_exam VARCHAR(255),
district VARCHAR(255),
requestor_name VARCHAR(255),
requestor_designation VARCHAR(255),
jmo_name VARCHAR(255),
lab_request_id VARCHAR(50),
pdf_url VARCHAR(500),
status VARCHAR(50) DEFAULT 'draft', -- 'draft', 'submitted'
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
created_by VARCHAR(50) REFERENCES users(id)
);

-- 8. PMR Identifiers Table (Child table for PMR Form)
CREATE TABLE pmr_identifiers (
    id SERIAL PRIMARY KEY,
    pmr_id VARCHAR(50) REFERENCES pmr_forms(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    address TEXT NOT NULL
);

-- 9. Lab Requests Table
CREATE TABLE lab_requests (
    id VARCHAR(50) PRIMARY KEY,
    patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,
    requested_by VARCHAR(50) REFERENCES users(id),
    requested_by_name VARCHAR(255),
    requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    form_type VARCHAR(50) NOT NULL, -- 'mlef', 'mlr', 'pmr'
    form_id VARCHAR(50) NOT NULL,
    test_types TEXT[], -- Array of test types e.g. ['toxicology',
'blood_alcohol']
    urgency VARCHAR(50) NOT NULL, -- 'routine', 'urgent', 'stat'
    clinical_history TEXT,
    status VARCHAR(50) DEFAULT 'pending', -- 'pending', 'in_progress',
'completed'
    test_results TEXT,
    observations TEXT,
    conclusion TEXT,
    lab_tech_name VARCHAR(255),
    completed_at TIMESTAMP
);

```

4.4 Constraints

The database utilises various constraints to ensure data integrity and enforce business rules.

- **Primary Keys:** Defined on every table to uniquely identify records. Examples:
 - id VARCHAR(50) PRIMARY KEY in users, patients, mlef_forms, mlr_reports, pmr_forms, lab_requests.
 - id SERIAL PRIMARY KEY in child tables like mlr_injuries and pmr_identifiers for auto-incrementing integer IDs.
- **Foreign Keys:** Used to establish relationships between tables and enforce referential integrity. All foreign keys referencing main entities include ON DELETE CASCADE to ensure related records are removed if a parent record is deleted. Examples:
 - patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE in mlef_forms, mlr_reports, pmr_forms, and lab_requests.
 - created_by VARCHAR(50) REFERENCES users(id) in form tables to track authorship.
 - mlr_id VARCHAR(50) REFERENCES mlr_reports(id) ON DELETE CASCADE in child tables like mlr_injuries.
- **UNIQUE:** Ensures column values are distinct across the table. Examples:
 - email VARCHAR(255) UNIQUE NOT NULL in the users table.
 - nic VARCHAR(50) UNIQUE NOT NULL in the patients table.
- **DEFAULT:** Assigns a default value if none is provided during an insert. Examples:
 - email_verified BOOLEAN NOT NULL DEFAULT FALSE in the users table.
 - status VARCHAR(50) DEFAULT 'draft' in form tables (mlef_forms, mlr_reports, pmr_forms).
 - created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP across most tables.
- **NOT NULL:** Implicitly applied via PRIMARY KEY and explicitly declared for mandatory fields (e.g., name VARCHAR(255) NOT NULL in users).

4.5 Indexes

Indexes have been created to optimise query performance, particularly for foreign key columns frequently used in joins and filtering.

```
-- Performance & Referential Integrity Indexes
CREATE INDEX IF NOT EXISTS idx_mlef_forms_patient_id ON mlef_forms(patient_id);
CREATE INDEX IF NOT EXISTS idx_mlef_forms_created_by ON mlef_forms(created_by);
CREATE INDEX IF NOT EXISTS idx_mlr_reports_patient_id ON
mlr_reports(patient_id);
CREATE INDEX IF NOT EXISTS idx_mlr_reports_created_by ON
mlr_reports(created_by);
CREATE INDEX IF NOT EXISTS idx_pmr_forms_patient_id ON pmr_forms(patient_id);
```

```
CREATE INDEX IF NOT EXISTS idx_pmr_forms_created_by ON pmr_forms(created_by);
CREATE INDEX IF NOT EXISTS idx_lab_requests_patient_id ON
lab_requests(patient_id);
CREATE INDEX IF NOT EXISTS idx_lab_requests_requested_by ON
lab_requests(requested_by);
CREATE INDEX IF NOT EXISTS idx_mlr_injuries_mlr_id ON mlr_injuries(mlr_id);
CREATE INDEX IF NOT EXISTS idx_mlr_grievous_entries_mlr_id ON
mlr_grievous_entries(mlr_id);
CREATE INDEX IF NOT EXISTS idx_pmr_identifiers_pmr_id ON
pmr_identifiers(pmr_id);
```

4.6 Views

The system does not include explicit SQL statements for creating Views. Data retrieval is currently handled via direct table queries or joins in the application logic.

4.7 Stored Procedures

The system does not include explicit SQL statements for Stored Procedures. Business logic and transactions are handled within the Node.js application layer.

4.8 Triggers

The system does not include explicit SQL statements for Triggers. Actions like cascading deletes are handled via Foreign Key constraints (ON DELETE CASCADE), and default timestamps are handled via DEFAULT CURRENT_TIMESTAMP.

4.9 Sample Data

Sample data is populated using the seed.js script to provide a working dataset for development and testing.

Sample User Record (Dr. Amal Perera):

```
{
  "id": "d1",
  "name": "Dr. Amal Perera",
  "role": "doctor",
  "designation": "Medical Officer",
  "email": "dr.perera@forensic.gov",
  "phone": "0771234567",
  "email_verified": true
}
```

Sample Patient Record (Ruwan Kumara):

```
{
  "id": "P2026-001",
  "name": "Ruwan Kumara",
  "dob": "1985-03-15",
  "sex": "Male",
}
```

```
"address": "123 Galle Road, Colombo 03",
"nic": "851530423V",
"email": "ruwan.k@gmail.com",
"phone": "0711234567",
"registered_by": "Nimal Silva",
"registered_at": "2026-06-20T08:30:00Z"
}
```

Sample MLEF Form Record:

```
{
  "id": "MLEF-2026-001",
  "patientId": "P2026-001",
  "policeStation": "Colombo Fort",
  "mlefNo": "CF/2026/1234",
  "examineeName": "Ruwan Kumara",
  "reasonForReferring": "Assault causing bodily harm – section 314 of Penal Code",
  "officerName": "Nimal Silva",
  "bodyHarmTypes": ["laceration", "contusion", "abrasion"],
  "causativeWeapon": ["blunt"],
  "hurtCategory": "non-grievous",
  "examFindings": "Multiple lacerations over scalp (3cm × 0.5cm)...",
  "doctorName": "Dr. Amal Perera",
  "status": "complete"
}
```

Sample Lab Request Record:

```
{
  "id": "LAB-2026-001",
  "patientId": "P2026-002",
  "requestedBy": "d2",
  "requestedByName": "Dr. Saman Fernando",
  "formType": "mlef",
  "formId": "MLEF-2026-001",
  "testTypes": ["blood_alcohol", "drug_screen"],
  "urgency": "urgent",
  "clinicalHistory": "Patient brought by police. Alleged assault. Suspected alcohol consumption.",
  "status": "pending"
}
```

Chapter 5 – System Development

5.1 System Architecture

The system follows a modern Client-Server architecture. The frontend is built as a Single Page Application using React, which provides a highly responsive user interface. The backend is built using Node.js and Express, exposing a RESTful API to handle business logic and data processing. A robust PostgreSQL relational database is used for persistent data storage. The frontend and backend communicate securely via HTTP requests using JSON format.

5.2 Login Module

The login module acts as the secure gateway to the application. Users must authenticate using their registered email address and password. The system checks the credentials against securely hashed passwords in the database. Additionally, the module ensures that the user has verified their email address before allowing their first login. It also features a secure password reset mechanism utilising emailed verification codes.

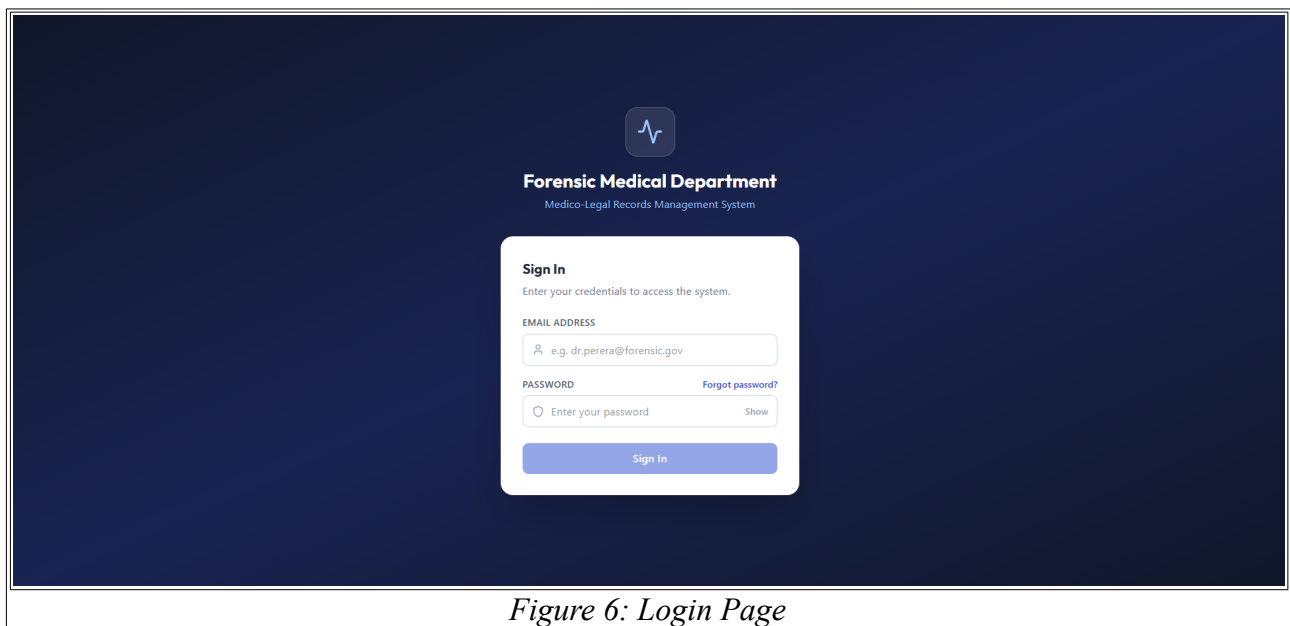


Figure 6: Login Page

5.3 Patient Management

This module allows medical and administrative staff to register and manage subjects (both living patients and deceased individuals). It captures essential demographic details such as Name, Date of Birth, Sex, Address, and NIC number. The system strictly enforces NIC uniqueness to prevent duplicate records. The patient profile acts as the central anchor to which all subsequent medical forms, cases, and lab requests are linked.

The screenshot shows the 'Register New Patient' form in the FMD System. The left sidebar contains navigation links: Dashboard, Register Patient (highlighted), Patient Records, Staff Management, MLEF Forms, MLR Reports, PMR Reports, Lab Reports, Daily Case Report, and Monthly Statistics. The user is Sanindu Hansara, Chief Admin Officer. The form fields include: PROFILE PICTURE (Upload/Choose File), FULL NAME (Full name), NIC NUMBER (e.g. 901234567V), DATE OF BIRTH (dd/mm/yyyy), SEX (Male), EMAIL ADDRESS (e.g. patient@example.com), PHONE NUMBER (e.g. 0712345678), and ADDRESS (Full residential address). A note states: '* Required. Patient ID auto-generated on registration.' Buttons for 'Cancel' and 'Register Patient' are at the top right.

Figure 7: Patient Registration Page

5.4 Case Management

Dedicated React forms allow doctors and JMOs to input histories and examination findings. These complex digital forms mimic the physical government forms. All of these forms can be accessed and viewed through the Patient Records page.

The screenshot shows the 'Patient Records' page in the FMD System. The left sidebar is the same as in Figure 7, with 'Patient Records' highlighted. The user is Sanindu Hansara, Chief Admin Officer. The page title is 'Patient Records' with a subtitle 'All registered patients and their associated forms'. A search bar is present. Two patient records are displayed: 1. 'test' (p-2026-10005, Male, yrs, NIC: 22234322111, Email: 2@mail.com, Phone: 2221232123, Registered 21/07/2026 by Sanindu Hansara). 2. 'Prabod' (p-2026-10004, Male, yrs, NIC: 123456789V, Email: pjg@gmail.com, Phone: 0714185280, Registered 25/06/2026 by Sanindu Hansara). Each record shows a 'CLINICAL SECTION' with MLEF, MLR, and Lab Requests. The 'test' record has MLEF 'MLEF-2026-10004' marked 'Complete'. The 'Prabod' record has MLEF '123556' marked 'Complete', MLR 'MLR-2026-10000' marked 'Submitted', and Lab 'LAB-2026-10000' marked 'Completed'. Buttons for '+ New MLEF' are present for each record. A '+ Register Patient' button is at the top right.

Figure 8: Patient Records Page

5.5 Postmortem Module

Dedicated to Judicial Medical Officers, this module manages Post-Mortem Reports for deceased subjects. It records forensic data related to the deceased person's body and provides a structured flow to document the cause of death officially.

FMD System
Medico-Legal Records

Dashboard
Patients
MLEF Forms
MLR Reports
PMR Reports
Lab Requests
Daily Case Report
Monthly Statistics

Dr. Rashmika Gunaratne
Judicial Medical Officer

Sign Out

New PMR Form

Download Documents Request Lab Report Save PMR Report

SELECT DECEASED PATIENT *
— Choose a patient —

Case & Inquest Details

INQUEST NO. CASE NO.
e.g. INQ/2026/89 e.g. CR/556/26

PLACE OF INQUEST COURTS
e.g. Magistrate Court Colombo

DATE OF INQUEST/REQUEST
22/07/2026

Deceased Particulars

NAME OF DECEASED PERSON DATE & TIME OF DEATH (IF KNOWN)
dd/mm/yyyy --:-- --

Post-Mortem Examination Details

MEDICAL OFFICER CONDUCTING POST-MORTEM DATE & TIME OF POST-MORTEM

Figure 9: PMR Form Page

5.6 Evidence Management

Evidence and test tracking is managed through the Laboratory Requests module. When evidence (blood/swabs) is collected, the doctor initiates a lab request linked directly to the patient's case form. The module tracks the urgency of the test, and maintains a chain of custody by recording which doctor requested the test and which lab technician processed and updated the final results.

FMD System
Medico-Legal Records

Dashboard
Lab Requests

Tharindu Abeysekera
Lab Assistant

Sign Out

Lab Reports
All pending and completed requests

Search by ID, patient name, patient ID, status or urgency...

LAB-2026-10000
Prabod - p-2026-10004
By Dr. Thimira De Silva - 30/06/2026, 20:14:00
URGENT Completed

Blood Alcohol Toxicology Blood Group Serology Trace Evidence Drug Screen Histology Urine Analysis DNA

"critical"

Report — MLT. Dissanayaka
Results: afafafa
Observations: afafafa
Conclusion: done, nice job
Completed: 30/06/2026, 20:16:10
Download PDF

LAB-2026-7213
qweqw - p-2026-6590
By Sanindu Hansara - 24/06/2026, 01:58:39
ROUTINE Completed

Histology

Report — Sanindu Hansara
Results: hkhllln
Observations:
Conclusion:
Completed: 24/06/2026, 02:14:33
Download PDF

Figure 10: Lab Records Page

5.7 Court Report Module

This module handles the finalisation of the Medico-Legal Report (MLR), which is the official document despatched to the courts. It allows doctors to summarise clinical findings into specific legal categories (e.g., grievous vs. non-grievous injuries, blunt weapon trauma, firearm injuries). It ensures that medical observations are translated into a standardised, legally admissible format.

FMD System
Medico-Legal Records

MLR — MLR-2026-10000
Patient: Prabod · p-2026-10004

Download Documents Generate Report PDF Save Report

Lab Report Requested · Completed

Patient Information

Name: Prabod ID: p-2026-10004 NIC: 123456789V
Age/Sex: yrs / Male Address: Galketiya, kalawana

C. Injuries — Nature, Size, Shape, Disposition and Site

No.	Nature, size, shape, disposition and site of injury
1	slight cut through heart
2	deep hole in head

+ Add Injury Row

SPECIAL INVESTIGATIONS (X-RAY, ETC.)

MRI Requested

D. Opinion

1. NON-GRIEVOUS INJURIES — SELECT INJURY NUMBERS

Dr. Rashmika Gunaratne
Judicial Medical Officer

Sign Out

Figure 11: MLR Page

5.8 Staff Management

An administrative module restricted to Admin users. It allows for the centralised registration and management of all hospital and forensic staff. The admin inputs the staff member's details, assigns a specific job designation, and grants them a system Role (Admin, Doctor, JMO, or Lab Technician), which dictates their access permissions across the system.

FMD System
Medico-Legal Records

Staff Management
Manage doctors, lab technicians and admin accounts

+ Register Staff

Doctors / Medical Officers 7 members

T	Thimira Dilshan AI analyst	Doctor	prabod327@gmail.com Phone: 074185280
S	Sanindu Hansara Medical Officer	Doctor	saninduhansara@gmail.com Phone: 0740318581
N	Nimal Medical Officer	Doctor	moviesfan332@gmail.com Phone: 65656755878
D	Dr. Nethmi Wickramasinghe Medical Officer	Doctor	dr.nethmi@forensic.gov Phone: 0771010101
D	Dr. Jeewantha Consultant	Doctor	prabodsc@gmail.com Phone: 0779259825
S	Sanindu Hansara Senior Medical Officer	Doctor	sanindu@doctor.com Phone: 0989765712
D	Dr. Kavindi Perera Medical Officer	Doctor	dr.kavindi@forensic.gov Phone: 0779998888

Judicial Medical Officers (JMO) 3 members

D	Dr. Rashmika Gunaratne Judicial Medical Officer	Judicial Medical Officer	jmo.rashmika@forensic.gov Phone: 0772020202
---	--	--------------------------	--

Sanindu Hansara
Chief Admin Officer

Sign Out

Figure 12: Staff Management Page

5.9 Report Generation

The system features automated report generation to eliminate manual paperwork.

- Document Export: Core forms like the MLEF, PMR, and MLR can be automatically formatted and exported as printable PDF documents for physical archiving and court submissions.
- Statistical Reports: The system aggregates database records to generate daily activity logs and monthly statistical charts, aiding hospital administration in tracking case volumes and lab workloads.

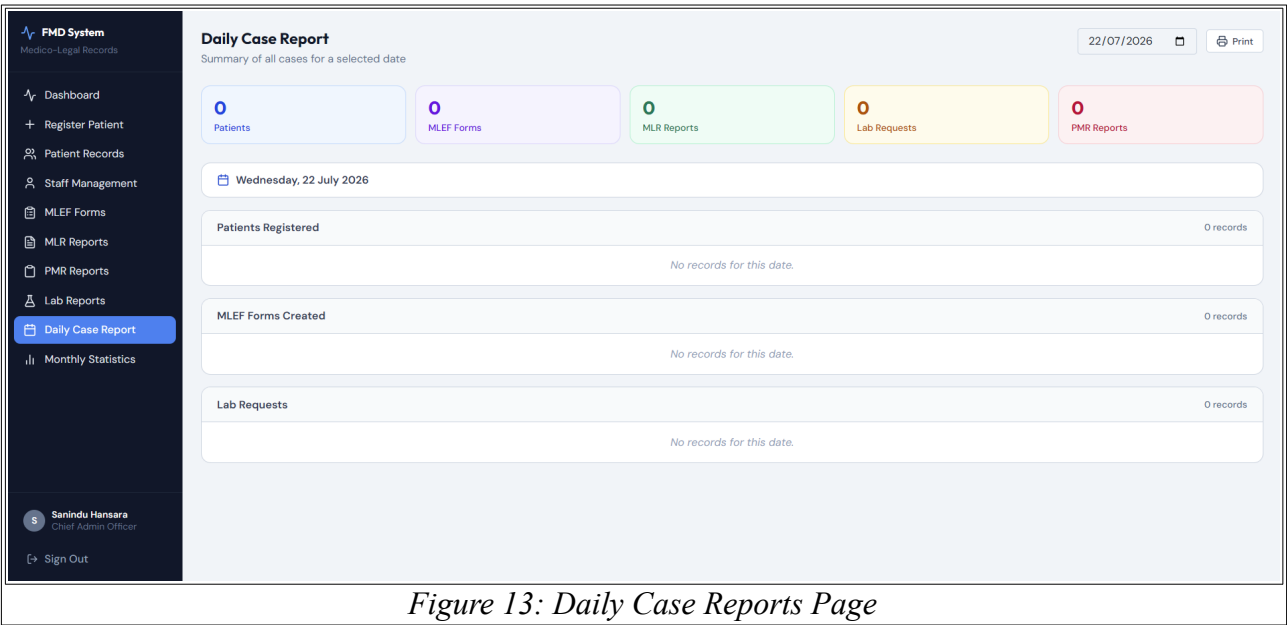


Figure 13: Daily Case Reports Page

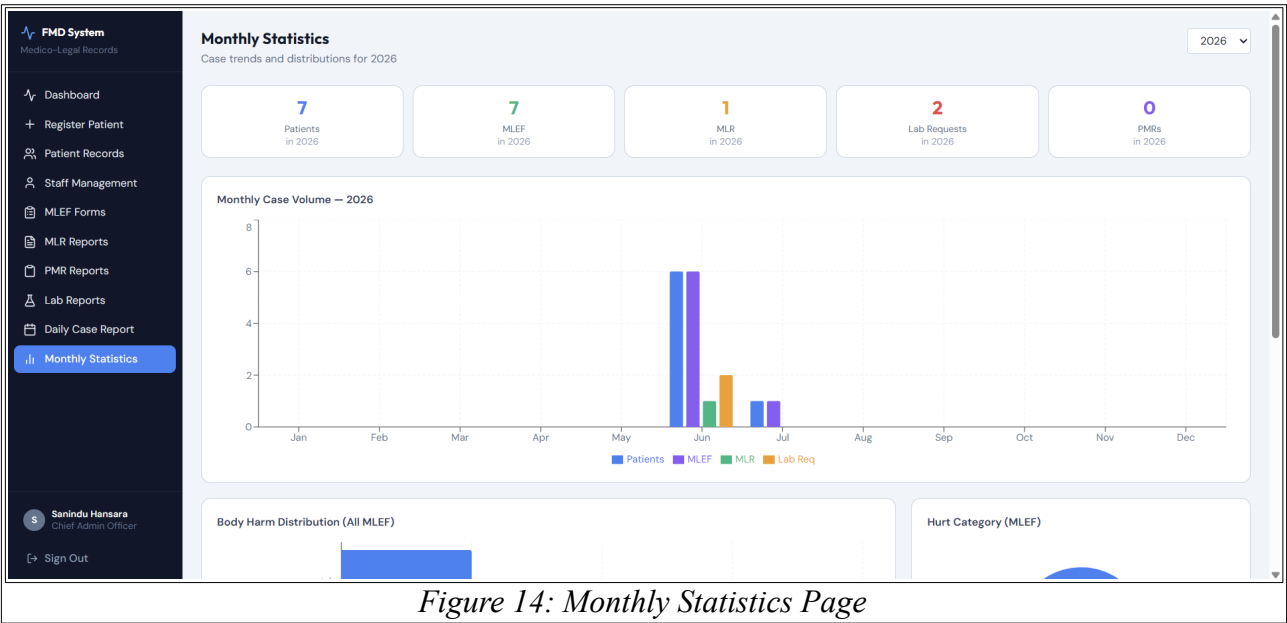


Figure 14: Monthly Statistics Page

5.10 Dashboard

Upon logging in, users are greeted with a role-specific digital dashboard. It acts as the central hub, providing immediate alerts for pending tasks. For example, a doctor's dashboard highlights active MLEF drafts and alerts them when lab results are ready, while an admin's dashboard displays high-level system statistics and recent staff registrations.

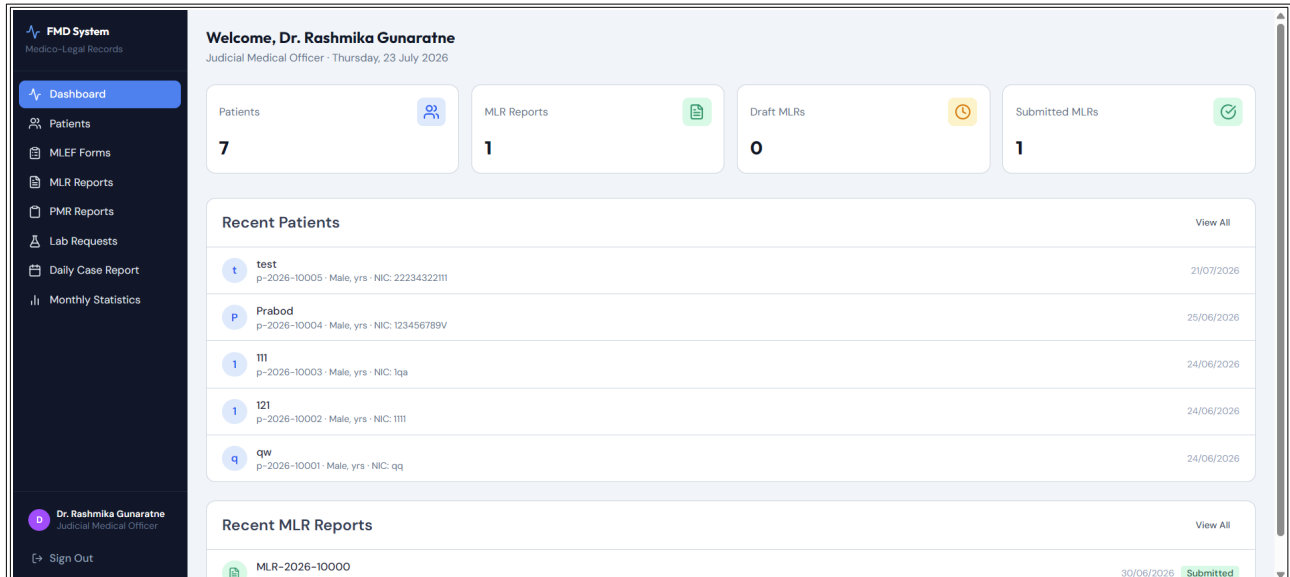


Figure 15: Dashboard Page

5.11 User Authentication

- **Login:** The authentication process verifies the user's email and password. Passwords are not stored in plaintext; the system compares the input against a bcrypt hash. Upon successful verification, the system issues a secure session token to keep the user authenticated during their session.
- **User Roles:** The system employs four distinct roles defined in the database: Administrative Officer, Doctor, Judicial Medical Officer, and Lab Technician.
- **Access Control:** Role-Based Access Control (RBAC) is strictly enforced. The user's role dictates their permissions. For instance, only an admin can access the Staff Management module. Only a lab technician can update test results in the Evidence Management module. Doctor and JMO roles have access to Patient and Case management, but cannot alter administrative settings. This ensures data confidentiality and operational integrity.

Chapter 6 – Testing

6.1 Test Plan

- **Testing Framework:** The testing suite is built using the Jest framework and Supertest to evaluate API controllers and routes.
- **Environment Configuration:** The environment is explicitly set to test mode using `process.env.NODE_ENV = "test"`.
- **Authentication Mocking:** Security and authorisation are tested using JSON Web Tokens (JWT), configured with a specific test secret key (`JWT_SECRET_KEY = "test-secret-key-for-jest"`).
- **Database Mocking:** To prevent test cross-contamination and isolate the API layer, database operations are mocked using `jest.spyOn` to intercept model methods.
- **Test Isolation:** Original methods are restored after each test using `jest.restoreAllMocks()` to ensure clean environments for subsequent tests.

6.2 Test Cases

Table 12: Test Cases

Test Case	Expected Result	Actual Result	Status
POST /api/auth/login Login with correct credentials	Returns status 200, a JWT token, and user object	Returns status 200, a JWT token, and user object	Pass
POST /api/auth/login Login with incorrect password	Returns status 401 with "Invalid email or password" message	Returns status 401 with "Invalid email or password" message	Pass
POST /api/auth/register Register user without admin token	Returns status 403 with "Access denied. Admins only." message	Returns status 403 with "Access denied. Admins only." message	Pass
GET /api/lab Access without authorisation	Returns status 401	Returns status 401	Pass
POST /api/lab Create lab request successfully	Returns status 201 and the created lab request data	Returns status 201 and the created lab request data	Pass
GET /api/mlcf/:id	Returns status 404 with	Returns status 404	Pass

Request non-existent MLEF form	"MLEF form not found" message	with "MLEF form not found" message	
POST /api/patients Register patient with missing fields	Returns status 400 with "All fields are required" message	Returns status 400 with "All fields are required" message	Pass
POST /api/pmr Save new PMR report	Returns status 200 and the saved input data	Returns status 200 and the saved input data	Pass

6.3 Validation Testing

- **Missing Field Validation:** Submitting incomplete data is caught by the system, returning a 400 Bad Request status with the message "All fields are required".
- **Authentication & Role Validation:** Routes validate user access levels; accessing admin-only routes (like user registration) with a standard doctor token triggers a 403 Access denied error.
- **Duplicate Entry Validation:** Attempting to register an account with an email that already exists in the system is rejected with a 400 status and a "User already exists" message.
- **Password Constraint Validation:** When resetting a password, the system validates length requirements; passwords under 8 characters are rejected with a 400 status.
- **Verification Code Validation:** Submitting incorrect or expired email verification or password reset codes is validated and rejected with 400 statuses, and the system tracks incorrect attempts to trigger a lockout (429 status) if too many occur.

6.4 SQL Query Testing

The database layer utilises parametrised SQL queries via a PostgreSQL connection pool to prevent SQL injection. Testing at this layer ensures data integrity, correct relationship mapping, and accurate data retrieval.

- **SELECT**
 - **Query Example:**

```
SELECT * FROM users WHERE email = $1;
```

- **Testing Focus:** Verifies that single-record fetching operates correctly and that user lookups by email return exactly one match or null, ensuring authentication controllers receive accurate user data.

- **JOIN**

- **Query Example:**

```
SELECT lr.*, COALESCE(u.name, lr.requested_by_name) as requested_by_name
FROM lab_requests lr LEFT JOIN users u ON u.id = lr.requested_by;
```

- **Testing Focus:** Tests the LEFT JOIN to ensure that Lab Requests successfully pull the requesting doctor's name from the users table. Validates that the fallback COALESCE function correctly handles cases where a user ID might be null or orphaned.
- **Aggregate Functions**
 - **Query Example:**

```
json_agg(json_build_object('id', i.id, 'no', i.injury_no, 'description', i.description))
```
 - **Testing Focus:** Ensures that PostgreSQL accurately aggregates multiple child rows (e.g., MLR injuries or PMR identifiers) into a single, structured JSON array, allowing complex hierarchical data to be returned in a single query.
- **Subqueries**
 - **Query Example:**

```
(SELECT json_agg(...) FROM mlr_injuries i WHERE i.mlr_id = m.id) as injuries
```
 - **Testing Focus:** Validates correlated subqueries used in fetching MLR and PMR records. Tests confirm that the subquery strictly matches i.mlr_id to the parent report's m.id, preventing cross-contamination of injury data between different patient reports.
- **GROUP BY**
 - **Note on Implementation:** The core CRUD queries rely on correlated subqueries and json_agg rather than explicit GROUP BY clauses to format parent-child relationships.
- **Views**
 - **Note on Implementation:** The current data models query base tables directly.

6.5 Backup and Recovery

- **Primary Cloud Backups (Supabase):** The system relies on Supabase's automated backup functions. This includes routine daily logical backups and Point-in-Time Recovery (PITR) to protect against accidental data deletion or corruption.
- **Secondary Local Backups:** A scheduled local cron job utilises PostgreSQL's native pg_dump utility to extract full database snapshots (schema and data). These encrypted dumps are stored securely on local infrastructure to provide an independent failover mechanism outside of the cloud provider.

Chapter 7 – Reports

Forensic Medical Department

Medico-Legal Records Management System

MEDICO-LEGAL EXAMINATION FORM

MLEF-2026-10003
Generated 23/07/2026, 02:58:10

PATIENT DETAILS

Patient ID	p-2026-10004	Full Name	Prabod
Date of Birth	23/05/2002	Sex	Male
NIC	123456789V	Phone	0714185280
Address	Galketiya, kalawana	Registered By	Sanindu Hansara

PART A — POLICE / ADMINISTRATIVE DETAILS

Police Station	Pera	MLEF No.	123556
Date of Issue	25/06/2026	Examinee Name	Prabod
Examinee Age	24	Examinee Sex	Male
Examinee Address	Galketiya, kalawana	Reason for Referring	Mental issue
Officer Name	Thimira	Rank	IGP
Reg. No.	135	Officer Station	Colombo 7
Part A Filled By	Sanindu Hansara	Part A Filled At	25/06/2026 08:50

PART B — MEDICAL OFFICER

Hospital	Kalawana	Ward	14
BHT No.	12	Place of Examination	On the bed
Admission Date	25/03/2026	Examination Date/Time	25/03/2026 17:42
Discharge Date	—		

EXAMINATION FINDINGS

Body Harm Types	Bite, Stab	Causative Weapon	Sharp, Grip/Teeth
Weapon (Other)	—	Hurt Category	fatal
Endangers Life	yes	Alcohol Examination	negative
Drugs Examination	negative	Signs of Sexual Assault	—
Internal Injuries	Mind injured		
Brief History	Frequently harrassed by Thimira		

CONFIDENTIAL — Medico-legal document. Unauthorised disclosure is prohibited.
Downloaded by Dr. Nethmi Wickramasinghe

Page 1 of 2

Figure 16: MLEF Form Report

Forensic Medical Department

Medico-Legal Records Management System

PMR-2026-10000

Generated 23/07/2026, 03:05:08

POST-MORTEM REPORT

PATIENT DETAILS

Patient ID	p-2026-10005	Full Name	test
Date of Birth	02/07/2026	Sex	Male
NIC	22234322111	Phone	2221232123
Address	1111	Registered By	Sanindu Hansara

INQUEST DETAILS

Inquest No.	INQ/23/45	Case No.	CR/34/56
Place	Piliyandala	Courts	Piliyandala Court
District	Kandy	Date	25/07/2026

DECEASED

Name of Deceased	test	Date/Time of Death	23/07/2026 18:07
------------------	------	--------------------	------------------

IDENTIFICATION OF THE BODY

Identifier Name	Address
Mama	23/32, M, City

EXAMINATION

Doctor Conducting	Dr. Rashmika Gunaratne	Date/Time of Examination	23/07/2026 22:11
Place of Examination	Here	Status	submitted

REQUESTED BY

Requestor Name	Test Suilva	Designation	TESTER
----------------	-------------	-------------	--------

Dr. Rashmika Gunaratne

Judicial Medical Officer

Here

Examination: 23/07/2026 22:11

Figure 17: PMR Form Report

Forensic Medical Department

Medico-Legal Records Management System

MLR-2026-10000

Generated 23/07/2026, 02:58:24

MEDICO-LEGAL REPORT**PATIENT DETAILS**

Patient ID	p-2026-10004	Full Name	Prabod
Date of Birth	23/05/2002	Sex	Male
NIC	123456789V	Phone	0714185280
Address	Galketiya, kalawana	Registered By	Sanindu Hansara

INJURIES OBSERVED

Injury No.	Description
1	slight cut through heart
2	deep hole in head

GRIEVOUS INJURIES

Injury No.	Limb / Region	Remarks
2	top of the head	it was deep

Non-Grievous Injury Nos.	1	Injuries Capable of Causing Death	heart injury
--------------------------	---	-----------------------------------	--------------

CATEGORISATION BY WEAPON TYPE

Blunt Weapon Nos.	—	Blunt Contusion Nos.	—
Cut Nos.	—	Sharp Cutting Nos.	—
Stab Nos.	—	Firearm Nos.	—
Burns Nos.	—	Bite Nos.	—

TOXICOLOGY & OBSERVATIONS

Smell of Liquor	yes	Under Influence of Liquor	yes
-----------------	-----	---------------------------	-----

Special Investigations	MRI Requested
------------------------	---------------

Further Notes	—
---------------	---

Figure 18: MLR Form Report

Daily Case Report

21/07/2026

Print

Summary of all cases for a selected date

1

Patients

1

MLEF Forms

0

MLR Reports

0

Lab Requests

0

PMR Reports

Tuesday, 21 July 2026

Patients Registered

1 record

PATIENT ID	FULL NAME	AGE / SEX	NIC	REGISTERED BY	TIME
p-2026-10005	test	/ Male	22234322111	Sanindu Hansara	11:29

MLEF Forms Created

1 record

MLEF ID	MLEF NO.	PATIENT	POLICE PART	MEDICAL PART	STATUS
MLEF-2026-10004	—	test	Pending	✓ Complete	Complete

Lab Requests

0 records

No records for this date.

Figure 19: Daily Case Report

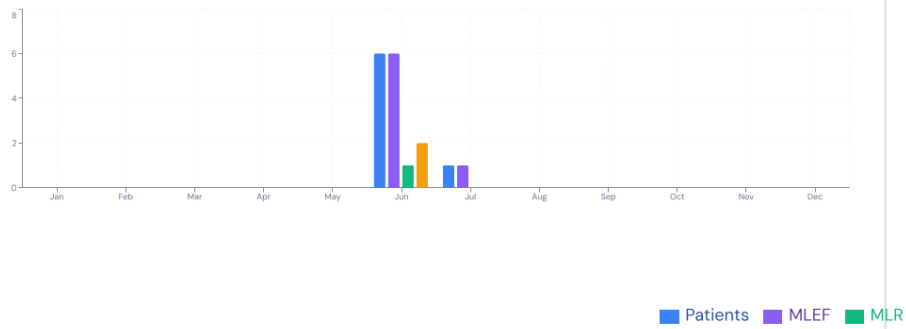
Monthly Statistics

Case trends and distributions for 2026

2026 ▼

7Patients
in 2026**7**MLEF
in 2026**1**MLR
in 2026**2**Lab Requests
in 2026**0**PMRs
in 2026

Monthly Case Volume — 2026



Body Harm Distribution (All MLEF)

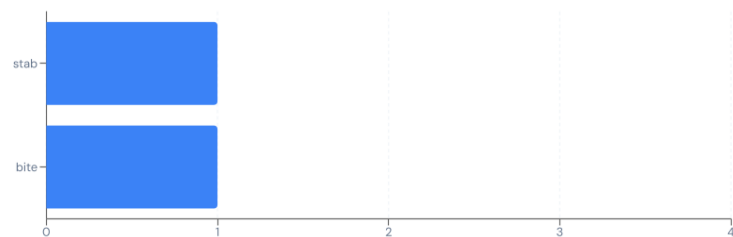


Figure 20: Monthly Statistics Report

Chapter 8 – Security

Password Security

The system prioritises the security of user credentials to prevent unauthorised access. Passwords are never stored in plaintext within the database. Instead, the application utilises the bcrypt hashing algorithm with a configurable salt round to securely hash passwords before storing them in the `password_hash` column of the users table. Furthermore, highly sensitive temporary codes, such as email verification and password reset codes, are also securely hashed rather than stored directly.

User Roles

The system operates on a clearly defined role hierarchy to differentiate system capabilities. Every staff member registered in the database is assigned one of four distinct roles, recorded in the `role` column of the users table:

- **admin:** Administrative staff
- **doctor:** Doctors handling clinical examinations
- **jmo:** Judicial Medical Officers specialising in forensic autopsies
- **lab:** Laboratory technicians handling evidence testing

Access Control

Access to modules and sensitive data is strictly governed by Role-Based Access Control (RBAC). When a user logs in, their session token verifies their role and restricts their navigation and data-modification privileges. For example:

- Only admin users can access the Staff Management module to register new accounts.
- Only doctor and jmo users can initiate and fill out medical forms or request lab tests.
- Only lab users have permission to update the status and input results for laboratory requests.

This ensures that users cannot accidentally or maliciously alter records outside their designated duties.

Confidentiality

Maintaining the confidentiality of patient demographics and forensic evidence is critical. The system enforces confidentiality through several layers:

- **Authentication & Authorisation:** Data is completely inaccessible without a verified login and appropriate role clearance.
- **Data Isolation:** A doctor's dashboard will prioritise cases relevant to them, hiding administrative data.

- **Secure Transmission:** HTTPS (SSL/TLS) is used to encrypt all data in transit between the frontend and the Node.js backend, preventing interception of sensitive medical records.

Audit Logs and Chain of Custody

While the system does not utilise a single, monolithic audit logs table, it successfully achieves detailed auditing and strict chain of custody through pervasive metadata tracking across all core tables.

- Every critical action records the "Who" and "When". For example, the MLEF form specifically tracks who filled out the police referral versus who conducted the clinical exam, alongside their respective timestamps.
- Similarly, the Laboratory module tracks the exact doctor who ordered a test and timestamps when the results were finalised, maintaining a legally defensible chain of custody for all evidence.

Database Backup and Recovery

- **Primary Cloud Backups (Supabase):** The system relies on Supabase's automated backup functions. This includes routine daily logical backups and Point-in-Time Recovery (PITR) to protect against accidental data deletion or corruption.
- **Secondary Local Backups:** A scheduled local cron job utilises PostgreSQL's native `pg_dump` utility to extract full database snapshots (schema and data). These encrypted dumps are stored securely on local infrastructure to provide an independent failover mechanism outside of the cloud provider.

Chapter 9 – Challenges and Future Improvements

Challenges

- **Requirement mapping:** Translating highly intricate physical medical documents into a normalised PostgreSQL database schema and responsive React components proved challenging. Ensuring every legal caveat was captured digitally required extensive requirement mapping.
- **PDF Generation:** Generating automated, browser-side PDFs that matched the required formatting and dimensions was a technical hurdle.
- **Interface Design & State Management:** Handling complex, multi-step forms in the frontend (especially for Postmortem reports where dozens of discrete findings must be recorded) required robust state management in React to ensure an intuitive interface and prevent data loss during input.
- **Database Complexity:** Designing a relational schema that accurately maps the relationships between a single patient and their multiple potential cases, varied injury types, and lab requests required careful normalisation to avoid data redundancy while maintaining query performance.

Future Improvements

- **Digital Signatures & Biometric Login:** Strict biometric logins could be implemented. Additionally, JMOs/doctors could be allowed to digitally sign MLRs and PMRs. This would solidify the chain of custody and eliminate the need to print and physically sign documents.
- **Mobile App Integration:** Developing a companion Mobile App would allow medical examiners to input notes or upload evidence photos directly from a secure tablet while at work.
- **Advanced Notification Systems (SMS & Email):** The existing functionality could be expanded to send automated email alerts to courts or police when an MLR is finalised. Additionally, integrating SMS notification alerts would also be helpful for internal staff.
- **QR Code Evidence Tracking:** A QR code generation module could be implemented for tagging physical evidence bags (like blood samples). These could be scanned by a mobile device to automatically open the linked digital lab request in the system.
- **AI-Assisted Reporting:** Intelligent tools could be integrated to assist doctors in summarising complex clinical findings quickly, or to automatically flag potential inconsistencies in recordings.

Chapter 10 – Conclusion

The development of the Medico-Legal Records Management System successfully achieved its primary objective of digitally transforming a complex, paper-based forensic registry into a fully operational web application. By engineering a highly normalised PostgreSQL database and a structured Node.js backend, the project successfully mapped intricate medico-legal entities – such as MLEF, MLR, and PMR forms – to specific role-based workflows for Admins, Doctors, JMOs, and Lab Technicians. Furthermore, the utilisation of a React and Vite frontend, combined with comprehensive Jest testing and Docker containerisation, ensured the delivery of a production-ready system built on a professional software engineering lifecycle.

The deployment of this system introduces profound operational benefits to the Forensic Medical Department, most notably transforming data retrieval times from hours to seconds through instant searchability. By enforcing strict Role-Based Access Control and cryptographic hashing, the application significantly enhances the security and confidentiality of highly sensitive forensic data. Additionally, automating the generation of standardised Medico-Legal and Post-Mortem reports drastically reduces administrative overhead and human error, ensuring that the courts receive accurate, uniformly formatted legal documents without delay.

Throughout the development lifecycle, several critical lessons were learned, particularly regarding the intersection of database design and legal compliance. A key realisation was the necessity of balancing strict normalisation with intentional denormalisation – such as storing historical user names directly on forms – to preserve the legal permanency required for court auditing. Additionally, the project reinforced the importance of separating and limiting permissions given to different user types and using robust automated testing when handling high stakes healthcare data. Finally, implementing the frontend highlighted that minimising cognitive load through clean, accessible interface design is crucial for medical officers tasked with accurately inputting complex clinical information under pressure.

References

1. C. Wickramasinghe, "Forensic Medicine Department Database System – Requirement Gathering Presentation," University of Peradeniya, 2026.
2. "PostgreSQL 16 Documentation," The PostgreSQL Global Development Group. [Online]. Available: <https://www.postgresql.org/docs/>
3. "Express – Node.js web application framework." [Online]. Available: <https://expressjs.com/>
4. "React – A JavaScript library for building user interfaces." [Online]. Available: <https://react.dev/>
5. "Node.js." [Online]. Available: <https://nodejs.org/>
6. "Vite – Next Generation Frontend Tooling." [Online]. Available: <https://vite.dev/>
7. "TypeScript – JavaScript with syntax for types." [Online]. Available: <https://www.typescriptlang.org/>
8. "Tailwind CSS – Rapidly build modern websites without ever leaving your HTML." [Online]. Available: <https://tailwindcss.com/>
9. "shadcn/ui – Beautifully designed components that you can copy and paste into your apps." [Online]. Available: <https://ui.shadcn.com/>
10. "Jest – Delightful JavaScript Testing." [Online]. Available: <https://jestjs.io/>
11. "Supabase – The Open Source Firebase Alternative." [Online]. Available: <https://supabase.com/>
12. "Docker Documentation." [Online]. Available: <https://docs.docker.com/>

Appendices

Appendix A – Complete SQL Schema Script (schema.sql)

```
-- PostgreSQL Database Schema for Medico-Legal Records Management System

-- 1. Users Table (Doctors, Admins, JMOs, Lab Techs)
CREATE TABLE users (
    id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL, -- 'doctor', 'jmo', 'admin', 'lab'
    designation VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(50) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    profile_picture_url VARCHAR(500),
    -- Email verification: a user must confirm their address once before first
login
    email_verified BOOLEAN NOT NULL DEFAULT FALSE,
    verification_code_hash VARCHAR(255), -- bcrypt hash of the 6-digit
code, never the code itself
    verification_expires_at TIMESTAMP,
    verification_sent_at TIMESTAMP, -- used to throttle resend
requests
    verification_attempts INTEGER NOT NULL DEFAULT 0,
    -- Password reset: same emailed-code mechanism as verification, kept in
-- separate columns so a reset in progress cannot clash with a verification
    reset_code_hash VARCHAR(255),
    reset_expires_at TIMESTAMP,
    reset_sent_at TIMESTAMP,
    reset_attempts INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Patients Table
CREATE TABLE patients (
    id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    dob DATE NOT NULL,
    sex VARCHAR(50) NOT NULL,
    address TEXT NOT NULL,
    nic VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    registered_by VARCHAR(255) NOT NULL, -- name of user
    profile_picture_url VARCHAR(500),
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. MLEF Forms (Medico-Legal Examination Forms)
```

```

CREATE TABLE mlef_forms (
  id VARCHAR(50) PRIMARY KEY,
  patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,

  -- Part A (Police / Admin)
  police_station VARCHAR(255),
  mlef_no VARCHAR(255),
  date_of_issue DATE,
  examinee_name VARCHAR(255),
  examinee_address TEXT,
  examinee_age VARCHAR(50),
  examinee_sex VARCHAR(50),
  reason_for_referring TEXT,
  officer_name VARCHAR(255),
  officer_rank VARCHAR(255),
  officer_reg_no VARCHAR(255),
  officer_police_station VARCHAR(255),
  part_a_filled_by VARCHAR(255),
  part_a_filled_at TIMESTAMP,

  -- Part B (Medical Officer)
  hospital VARCHAR(255),
  ward VARCHAR(255),
  bht_no VARCHAR(255),
  admission_date DATE,
  exam_date_time TIMESTAMP,
  discharge_date DATE,
  exam_place VARCHAR(255),
  body_harm_types TEXT[], -- Array of strings e.g. ['laceration',
'contusion']
  internal_injuries TEXT,
  causative_weapon TEXT[], -- Array of strings
  causative_weapon_other TEXT,
  hurt_category VARCHAR(50),
  endangers_life VARCHAR(50),
  alcohol_exam VARCHAR(50),
  drugs_exam VARCHAR(50),
  sexual_assault_signs TEXT[], -- Array of strings
  brief_history TEXT,
  exam_findings TEXT,
  investigations TEXT,
  referrals TEXT,
  other_opinions TEXT,
  remarks TEXT,
  doctor_name VARCHAR(255),
  doctor_qualifications VARCHAR(255),
  slmc_reg_no VARCHAR(255),
  doctor_designation VARCHAR(255),
  ref_no VARCHAR(255),
  part_a_pdf_url VARCHAR(500),
  part_b_pdf_url VARCHAR(500),
  part_b_filled_by VARCHAR(50) REFERENCES users(id),
  part_b_filled_at TIMESTAMP,

```

```

        lab_request_id VARCHAR(50), -- Linked lab request
        status VARCHAR(50) DEFAULT 'draft', -- 'draft', 'complete'
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        created_by VARCHAR(50) REFERENCES users(id)
    );

-- 4. MLR Reports (Medico-Legal Reports)
CREATE TABLE mlr_reports (
    id VARCHAR(50) PRIMARY KEY,
    patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,
    special_investigations TEXT,
    non_grievous_nos TEXT[], -- Array of injury numbers
    death_causing_count VARCHAR(255),
    blunt_weapon_nos VARCHAR(255),
    blunt_contusion_nos VARCHAR(255),
    cut_nos VARCHAR(255),
    sharp_cutting_nos VARCHAR(255),
    stab_nos VARCHAR(255),
    firearms_nos VARCHAR(255),
    burns_nos VARCHAR(255),
    bite_nos VARCHAR(255),
    further_notes TEXT,
    patient_smell_liquor VARCHAR(50),
    under_influence_liquor VARCHAR(50),
    doctor_name VARCHAR(255),
    doctor_qualifications VARCHAR(255),
    designation VARCHAR(255),
    station VARCHAR(255),
    date_of_despatch DATE,
    lab_request_id VARCHAR(50),
    pdf_url VARCHAR(500),
    status VARCHAR(50) DEFAULT 'draft', -- 'draft', 'submitted'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_by VARCHAR(50) REFERENCES users(id)
);

-- 5. MLR Injuries Table (Child table for MLR Report)
CREATE TABLE mlr_injuries (
    id SERIAL PRIMARY KEY,
    mlr_id VARCHAR(50) REFERENCES mlr_reports(id) ON DELETE CASCADE,
    injury_no VARCHAR(50) NOT NULL,
    description TEXT NOT NULL
);

-- 6. MLR Grievous Entries Table (Child table for MLR Report)
CREATE TABLE mlr_grievous_entries (
    id VARCHAR(50) PRIMARY KEY,
    mlr_id VARCHAR(50) REFERENCES mlr_reports(id) ON DELETE CASCADE,
    injury_no VARCHAR(50),
    limb VARCHAR(255),
    remarks TEXT
);

```

```

-- 7. PMR Reports (Post-Mortem Reports)
CREATE TABLE pmr_forms (
    id VARCHAR(50) PRIMARY KEY,
    patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,
    inquest_no VARCHAR(255),
    place VARCHAR(255),
    courts VARCHAR(255),
    date DATE,
    case_no VARCHAR(255),
    deceased_name VARCHAR(255),
    date_time_of_death TIMESTAMP,
    doctor_conducting VARCHAR(255),
    date_time_of_exam TIMESTAMP,
    place_of_exam VARCHAR(255),
    district VARCHAR(255),
    requestor_name VARCHAR(255),
    requestor_designation VARCHAR(255),
    jmo_name VARCHAR(255),
    lab_request_id VARCHAR(50),
    pdf_url VARCHAR(500),
    status VARCHAR(50) DEFAULT 'draft', -- 'draft', 'submitted'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_by VARCHAR(50) REFERENCES users(id)
);

-- 8. PMR Identifiers Table (Child table for PMR Form)
CREATE TABLE pmr_identifiers (
    id SERIAL PRIMARY KEY,
    pmr_id VARCHAR(50) REFERENCES pmr_forms(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    address TEXT NOT NULL
);

-- 9. Lab Requests Table
CREATE TABLE lab_requests (
    id VARCHAR(50) PRIMARY KEY,
    patient_id VARCHAR(50) REFERENCES patients(id) ON DELETE CASCADE,
    requested_by VARCHAR(50) REFERENCES users(id),
    requested_by_name VARCHAR(255),
    requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    form_type VARCHAR(50) NOT NULL, -- 'mlef', 'mlr', 'pmr'
    form_id VARCHAR(50) NOT NULL,
    test_types TEXT[], -- Array of test types e.g. ['toxicology',
'blood_alcohol']
    urgency VARCHAR(50) NOT NULL, -- 'routine', 'urgent', 'stat'
    clinical_history TEXT,
    status VARCHAR(50) DEFAULT 'pending', -- 'pending', 'in_progress',
'completed'
    test_results TEXT,
    observations TEXT,
    conclusion TEXT,
    lab_tech_name VARCHAR(255),

```

```

        completed_at TIMESTAMP
    );

-- 10. Performance & Referential Integrity Indexes
CREATE INDEX IF NOT EXISTS idx_mlef_forms_patient_id ON mlef_forms(patient_id);
CREATE INDEX IF NOT EXISTS idx_mlef_forms_created_by ON mlef_forms(created_by);
CREATE INDEX IF NOT EXISTS idx_mlr_reports_patient_id ON
mlr_reports(patient_id);
CREATE INDEX IF NOT EXISTS idx_mlr_reports_created_by ON
mlr_reports(created_by);
CREATE INDEX IF NOT EXISTS idx_pmr_forms_patient_id ON pmr_forms(patient_id);
CREATE INDEX IF NOT EXISTS idx_pmr_forms_created_by ON pmr_forms(created_by);
CREATE INDEX IF NOT EXISTS idx_lab_requests_patient_id ON
lab_requests(patient_id);
CREATE INDEX IF NOT EXISTS idx_lab_requests_requested_by ON
lab_requests(requested_by);
CREATE INDEX IF NOT EXISTS idx_mlr_injuries_mlr_id ON mlr_injuries(mlr_id);
CREATE INDEX IF NOT EXISTS idx_mlr_grievous_entries_mlr_id ON
mlr_grievous_entries(mlr_id);
CREATE INDEX IF NOT EXISTS idx_pmr_identifiers_pmr_id ON
pmr_identifiers(pmr_id);

```

Appendix B – Source Code Structure

```

Forensic-Medical-Project/
├── .git/                                # Git version control directory
├── docker-compose.yml                  # Docker orchestration configuration
├── Jenkinsfile                         # CI/CD pipeline configuration
├──
├── backend/                           # Node.js & Express API Server
│   ├── config/                        # Database connection configurations
│   ├── controllers/                   # Business logic controllers
│   ├── middlewares/                   # Custom middleware
│   ├── models/                       # Database SQL interactions
│   ├── routes/                       # API route definitions
│   ├── tests/                         # Automated backend test suites
│   ├── utils/                         # Helper functions
│   ├── node_modules/                 # Installed backend dependencies
│   ├── .dockerignore                 # Files to ignore in Docker build
│   ├── .env                          # Environment variables (Secrets)
│   ├── .env.example                  # Template for environment variables
│   ├── .gitignore                    # Files to ignore in Git
│   ├── app.js                        # Express app setup
│   ├── Dockerfile                    # Backend container configuration
│   ├── index.js                      # Server entry point
│   ├── init-db.js                    # Script to initialize database tables
│   ├── jest.config.js                # Configuration for Jest testing framework
│   ├── migrate-db.js                 # Script for database migrations
│   ├── package-lock.json             # Exact dependency versions
│   ├── package.json                  # Backend dependencies and scripts
│   ├── schema.sql                    # PostgreSQL database schema
│   └── seed-db.js                    # Script to populate sample data

```

└─ frontend/	# React Single Page Application (SPA)
└─ dist/	# Compiled production build output
└─ guidelines/	# Project/UI guidelines
└─ node_modules/	# Installed frontend dependencies
└─ src/	# Main source code directory
└─ app/	# Core application setup/routing
└─ components/	# Reusable UI components
└─ context/	# React Context for global state
└─ data/	# Static data or constants
└─ lib/	# Libraries and utilities
└─ pages/	# Top-level page components
└─ styles/	# Global CSS and styling files
└─ test/	# Frontend testing setup
└─ types/	# TypeScript type definitions
└─ main.tsx	# Frontend entry point
└─ .dockerignore	# Files to ignore in Docker build
└─ .env	# Frontend environment variables
└─ .gitignore	# Files to ignore in Git
└─ ATTRIBUTIONS.md	# Open-source attributions and licenses
└─ default_shadcn_theme.css	# Base styling theme file
└─ Dockerfile	# Frontend container configuration
└─ index.html	# Main HTML template file
└─ nginx.conf	# Web server configuration for production
└─ package-lock.json	# Exact dependency versions
└─ package.json	# Frontend dependencies and scripts
└─ pnpm-workspace.yaml	# Monorepo/workspace configuration
└─ postcss.config.mjs	# Configuration for PostCSS (CSS processing)
└─ README.md	# Frontend documentation
└─ vite.config.ts	# Configuration for Vite build tool

Appendix C – Docker Configuration

```

services:
  backend:
    build: ./backend
    container_name: forensic-backend
    restart: unless-stopped
    ports:
      - "3000:3000"
    env_file:
      - ./backend/.env

  frontend:
    build: ./frontend
    container_name: forensic-frontend
    restart: unless-stopped
    ports:
      - "80:80"
    depends_on:
      - backend

```

Individual Contribution

Table 13: Individual contributions

Student (Name and Reg No)	Role	Tasks Completed
H. P. J. Gunawardhana E/22/126	Team Leader	Documentation, coordination, submission and testing.
S. H. S. Hansara E/22/130	Web Interface Developer	Coded and deployed the website (frontend, backend, schemas)
T. H. Abeywickrama E/22/008	Database Designer	Designed ER and EER diagrams. Normalised schema. Prepared final report.
H. T. D. Hatharasinghe E/22/135	Requirement Analyst	Conducted requirement analysis, prepared SRS documentation and also implemented PDF downloading capabilities in website.

Project Timeline

Table 14: Project timeline

Week	Activity	Status
1	Requirement Analysis	Completed
2	Database Design	Completed
3	Interface Design	Completed
4	Database Implementation	Completed
5	Testing & Reports	Completed
6	Documentation & Presentation	Completed

Checklist Before Submission

- ✓ Requirement Analysis
- ✓ ER Diagram
- ✓ Relational Schema
- ✓ Normalization
- ✓ SQL Scripts
- ✓ Constraints
- ✓ Sample Data
- ✓ Front-end Screens
- ✓ SQL Queries
- ✓ Reports
- ✓ Testing
- ✓ Security
- ✓ Backup & Recovery
- ✓ Individual Contribution
- ✓ References